

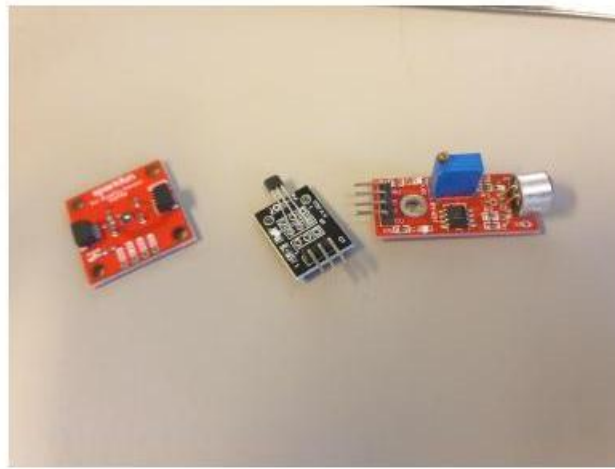
Sensors & Systems

Sensor Fusion

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

Section 1 What is Sensor Fusion?

1.1 Simple Sensors — A Starting Point



“Simple” sensors (From left: air quality, Hall effect, microphone)

- ▶ Measure one continuous physical quantity;
- ▶ Provide a simple analog or digital interface;
- ▶ Do not require extensive processing to obtain the desired information.

Figure 1: Three examples of “simple” sensors used throughout the course. From left: an **air quality sensor** (measures a single gas concentration), a **Hall effect sensor** (detects the presence and strength of a magnetic field), and a **microphone** (measures acoustic pressure as a voltage). Each device outputs *one* continuous physical quantity via a simple analog or digital interface.

Throughout this course we have built up the theory of state estimation — Kalman filters, EKF, UKF — largely in the context of a single sensor measuring a single physical quantity. Recall the defining properties of a “simple” sensor:

- **Measures one continuous physical quantity** — one number out per sample (temperature, pressure, light intensity, magnetic field strength, etc.).
- **Provides a simple analog or digital interface** — the output is directly readable as a voltage, a current, or a digital word. No significant onboard processing is needed.

- **Does not require extensive processing to obtain the desired information** — the measured quantity is already close to what the application needs. A microphone gives you the sound pressure; you do not need to do much more to know how loud something is.

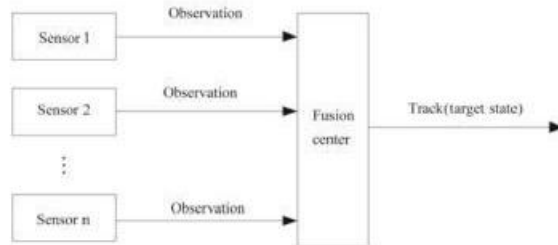
Why “simple” sensors are not always sufficient

Simple sensors have two fundamental weaknesses that motivate sensor fusion:

- **Each sensor measures something slightly different from what you ultimately want.** A gyroscope measures *angular rate*; you want *orientation*. A GPS receiver measures *position* but is noisy; an accelerometer measures *acceleration* but drifts. No single sensor gives you what you want, cleanly and completely.
- **Each sensor has its own noise, bias, and blind spots.** A gyroscope drifts over time (integrating a small bias error builds up into a large orientation error). A magnetometer is accurate in steady state but disturbed by nearby metal. An accelerometer cannot distinguish gravity from linear acceleration. These weaknesses are different for each sensor — which means they can complement each other if combined correctly.

The idea of sensor fusion is to use several sensors at once so that the strengths of each compensate for the weaknesses of the others. The remaining sections of this lecture develop three concrete frameworks for doing this: the Kalman filter (Section 2), the complementary filter (Section 3), and the Madgwick/Mahoney filters for attitude estimation (Sections 4–7).

1.2 Sensor Fusion — Definition and Goal



Sensor fusion is a broad term referring to using several sensors at the same time to provide information for a united purpose – e.g., estimating a common state.

Figure 2: General sensor fusion architecture. Each of n sensors produces an **observation** (a noisy measurement of some aspect of the environment or system state). All observations are fed into a central **fusion centre**, which combines them to produce a single estimate of the **track** or **target state** — the quantity we actually care about. The arrows indicate that information flows *from* the sensors *to* the fusion centre, not the other way around: the sensors do not communicate with each other, only with the central estimator.

Sensor fusion is a broad term referring to the use of several sensors simultaneously to provide information for a united purpose — most commonly, to estimate a shared underlying *state*.

The key word is **state**: sensor fusion is not simply about collecting more data. It is about combining multiple imperfect windows onto the same underlying physical reality to build a better estimate of that reality than any single window could provide.

What “estimating a common state” means — a concrete example

Suppose you want to know the *position and velocity* of a drone in 3D space. This is the **state**: the vector $[x, y, z, v_x, v_y, v_z]^\top$ that fully describes where the drone is and how fast it is moving at a given moment.

Now consider what individual sensors see:

- **GPS** gives you (x, y, z) — position only, updated at 1–10 Hz, with errors of several metres.
- **Accelerometer** gives you (a_x, a_y, a_z) — the sum of linear acceleration and gravity,

updated at ~ 1000 Hz, but drifts when integrated to position.

- **Barometer** gives you altitude z — slowly but steadily.
- **Camera** gives you optical flow — velocity relative to what the camera sees.

None of these directly gives the full state at the required accuracy and update rate. But together, fed into a fusion centre (e.g. a Kalman filter), they produce a position and velocity estimate that is better than any of them individually.

That is sensor fusion: the state $[x, y, z, v_x, v_y, v_z]^\top$ is the common thread; each sensor provides a noisy, partial view of it; the fusion centre combines all views into the best possible estimate.

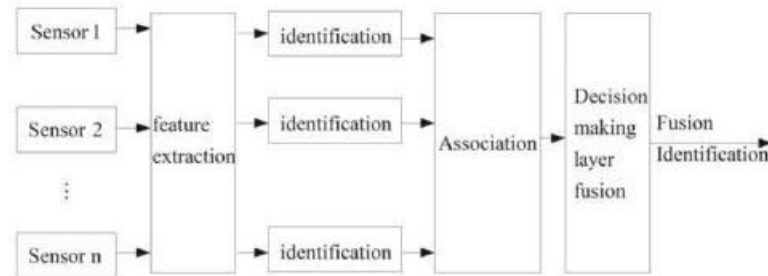
What makes one fusion approach better than another?

The fusion centre in the block diagram above is a placeholder. In reality it can be implemented in several ways, each with different assumptions and trade-offs:

- **Kalman filter** (Section 2): optimal when the system dynamics are linear and all noise is Gaussian. Gives the minimum-variance state estimate. Naturally handles multiple sensors with a single unified framework.
- **Complementary filter** (Section 3): a simpler, frequency-domain approach that splits the measurement bandwidth between sensors. Computationally cheap and robust, but less general than the KF.
- **Madgwick / Mahoney filters** (Sections 6–7): nonlinear filters specifically designed for attitude estimation from IMU data. They exploit the geometry of rotations (quaternions) and are very efficient to run on embedded hardware.

All three frameworks appear in this lecture. They differ in complexity, generality, and computational cost, but they all serve the same purpose: combining imperfect observations into one good estimate of the state.

1.3 Sensor Fusion for Decision Making



Decision making processes often make use of several different sensors to achieve more reliable feature detection – e.g., autonomous cars that rely on both radar, LiDAR, and cameras to detect obstacles, road signs etc.

Figure 3: Sensor fusion pipeline for **decision making**. Each sensor’s raw output first passes through **feature extraction** (pulling out relevant descriptors from the raw signal) and then its own **identification** stage (classifying what those features correspond to — a pedestrian, a road sign, a vehicle). The labelled detections from all sensors are then **associated** (matching detections across sensors that refer to the same real-world object), before the **decision making layer** combines the evidence into one agreed-upon classification that drives the system’s action.

This is a fundamentally different use of sensor fusion from state estimation. In state estimation (Section 2), you have a *known physical state* — position, velocity, orientation — and each sensor gives you a noisy measurement of that same quantity. The mathematics then tells you how to weight and combine those measurements optimally.

Here that is not the case at all. **Decision-making fusion** is about having several sensors that each form their own *independent opinion* about what is in the scene, and then making the sensors **agree** on a final answer. There is no state vector, no Kalman gain, no covariance matrix. Each sensor does its own full processing pipeline — feature extraction, identification, classification — and arrives at a symbolic label (“pedestrian with 87% confidence”, “road sign”, “car”). The fusion step then takes all those labels and resolves them into one committed decision.

A canonical example is the **autonomous car**:

- **Radar** processes its echo returns and concludes: “moving object, 30 m ahead, radial velocity 1.2m/s.” It cannot tell whether it is a pedestrian or a large cardboard box.
- **LiDAR** processes its point cloud and concludes: “upright shape, roughly human pro-

portions, 30 m ahead.” It cannot confirm the object is actually moving toward the road.

- **Camera** processes its image and concludes: “pedestrian bounding box detected, 90% confidence, at this pixel location.” It has no direct range measurement.

Each sensor arrives at its own answer independently. The **association** step then asks: do the radar’s “moving object at 30 m”, the LiDAR’s “human shape at 30 m”, and the camera’s “pedestrian at this pixel” all refer to the same real object? If yes, the **fusion identification** step combines the three independent opinions into one: “pedestrian confirmed, 30 m ahead, moving 1.2 m/s.” This combined verdict is far more reliable than any single sensor could produce, because the sensors would all have to make the same error simultaneously to produce a false positive — and since they use different physical principles, their errors are *independent*.

Why this is “agreeing” and not “averaging”

In state estimation fusion, you literally compute a weighted average of measurements. If GPS says you are at 57.00 °N and another sensor says 57.02 °N, the fused answer is somewhere between the two, weighted by each sensor’s reliability. You combine *numbers* about the *same physical quantity*.

In decision-making fusion you cannot do that. A radar range of 30 m and a camera pixel coordinate of (320, 240) are not two noisy measurements of the same thing — they are different types of information from different physical principles. There is no sensible way to “average” them.

Instead each sensor produces a *vote*: “I believe there is a pedestrian.” The fusion layer then decides whether the sensors agree enough to commit to that decision. If radar says “something is there” and LiDAR says “human-shaped” and the camera says “pedestrian”, all three votes point the same way and the system acts on it. If only the camera fires and the others see nothing, the system may withhold the decision until it has more evidence.

This is fundamentally a **consensus problem**, not an estimation problem. The tools used — voting rules, Dempster-Shafer theory, Bayesian classifiers, neural network ensembles — are different from the Kalman filter, and the result is a *label* or *decision*, not a state estimate with a covariance.

Why independent sensor errors make fusion so powerful here

The key reason to fuse sensors in a decision pipeline is that their failure modes are **independent**:

- Heavy rain scatters the LiDAR beam — but the radar penetrates rain effortlessly.
- Darkness blinds the camera — but LiDAR and radar are unaffected.
- A mirrored surface confuses LiDAR (returns go away from the sensor) — but the camera can still read a sign on that surface.

For the system to make a *wrong* decision, all the sensors that disagree must all fail in

the same direction at the same moment. The probability of that is the product of the individual failure probabilities — which is much smaller than any single sensor’s failure probability. This is the core reliability argument for multi-sensor decision systems.

1.4 Image Fusion — A Concrete Decision-Making Example

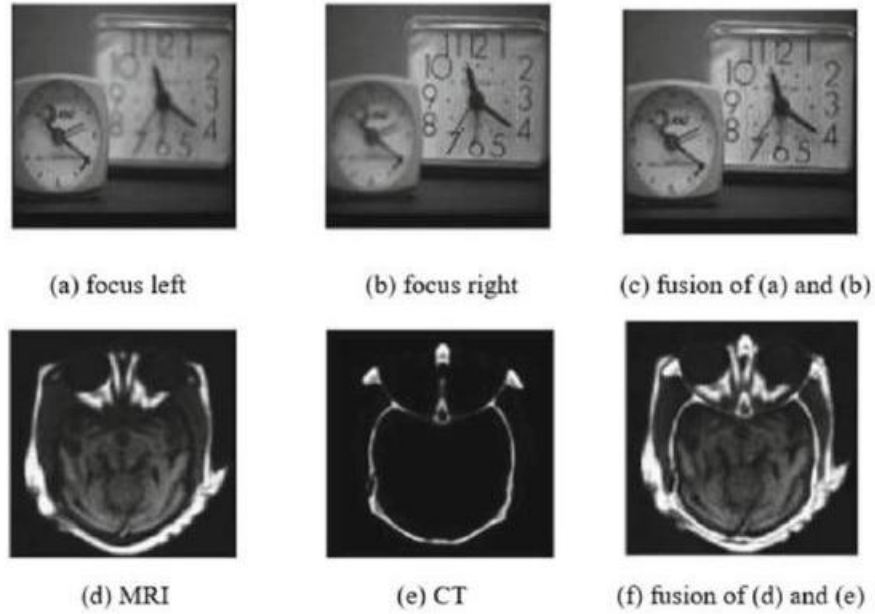


Figure 4: Two examples of image-level sensor fusion. **Top row:** optical focus fusion. (a) An image focused on the *left* clock — the right clock is blurred. (b) An image focused on the *right* clock — the left clock is blurred. (c) A fusion of (a) and (b) — both clocks are sharp simultaneously. No single camera position or focus setting achieves this; only fusing two images with complementary sharp regions produces the result. **Bottom row:** medical imaging fusion. (d) **MRI** image: excellent soft-tissue contrast (shows internal organs and neural structures), but poor bone visibility. (e) **CT** image: excellent bone and dense-structure contrast, but less soft-tissue detail. (f) Fusion of (d) and (e): both soft tissue and bone structure are visible simultaneously, giving the clinician more information than either modality alone.

Image fusion makes the complementary-sensor concept visually intuitive. In both examples, no single sensor can capture everything:

Optical focus fusion exploits a fundamental limitation of any lens-based imaging system: a camera can only be sharply focused at one distance at a time. Objects closer or further than the focal plane appear blurred. By taking two images with different focal distances and fusing them — keeping only the sharp regions from each — one obtains a single image where everything is in focus. This is the same complementary principle as the complementary filter of Section 3: each sensor (each image) contributes what it is best at (its in-focus region), and the fusion discards the rest.

MRI + CT fusion illustrates complementary physics:

- **MRI** (Magnetic Resonance Imaging) uses strong magnetic fields and radio waves to image the way hydrogen nuclei in water and fat relax. Tissues rich in water (brain, muscle, soft organs) give strong MRI signal; dense bone contains little free water and gives a weak signal. MRI is therefore excellent for soft tissue but poor for bone.
- **CT** (Computed Tomography) uses X-rays, which are absorbed in proportion to tissue density. Dense bone absorbs X-rays strongly and appears bright; soft tissue absorbs little and appears grey. CT is excellent for bone and dense structures but has lower soft-tissue contrast than MRI.

The two modalities are physically *complementary*: each sees what the other misses. Their errors are also independent — metal implants create artefacts in MRI but are visible on CT; fluid-filled cavities appear clearly on MRI but can be ambiguous on CT. Fusing both images gives the clinician information that would otherwise require two separate examinations.

Image fusion is harder than it looks — the registration problem

Before any pixel-level fusion can happen, the two images must be **registered**: they must be aligned so that each pixel in image (a) corresponds to the same physical location in image (b).

For the focus example this is trivial: the camera does not move between shots, so the pixels already align. For MRI + CT it is a major unsolved engineering challenge:

- The patient may move slightly between scans.
- The two machines have different geometric distortions and resolutions.
- MRI and CT are acquired at different times and may show different tissue states (e.g. bladder fill level).

Registration is typically done by finding a rigid-body or non-rigid transformation (translation, rotation, scaling, warping) that maximises some measure of similarity between the two images. Only after registration is the pixel-level fusion meaningful. This is an example of the **data association problem** from Section 1.3 — here at the pixel level rather than the object level.

Why these image examples are relevant to the rest of the lecture

The two image fusion examples are not just illustrations — they encode the two fundamental reasons to fuse sensors that appear throughout this lecture:

Reason 1: The same thing, measured twice, more accurately. The focus fusion example fuses two images of *the same scene*. Neither image is wrong — they are both correct pictures of the same reality, just with different blind spots (out-of-focus regions). Fusing them reduces those blind spots. This is the same motivation as the Kalman filter for sensor fusion (Section 2): both sensors measure the same state, each with its own noise, and combining them gives a better estimate than either alone.

Reason 2: Two different things, each inaccessible to the other sensor. The

MRI + CT example fuses two images that capture *physically different* aspects of the same patient. Each image contains information the other cannot — they are genuinely complementary. This is the same motivation as the complementary filter (Section 3) and the IMU fusion in Sections 4–7: the gyroscope sees high-frequency orientation changes that the accelerometer cannot track cleanly; the accelerometer sees the absolute gravity direction that the gyroscope cannot maintain over time. Both are needed.

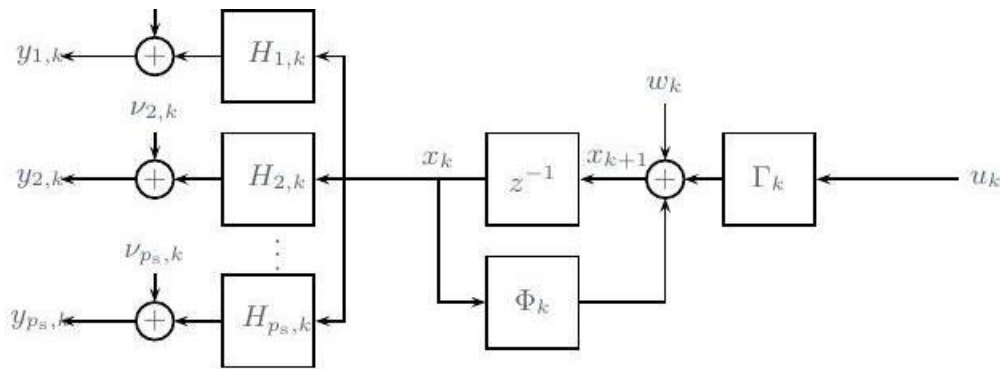
Section 1 — Key Takeaways

- **Simple sensors** each measure one physical quantity via a simple interface and require minimal processing. They are individually limited by noise, drift, and blind spots.
- **Sensor fusion** uses several sensors simultaneously to estimate a common state or make a joint decision, exploiting the fact that different sensors fail in different ways.
- There are two distinct fusion paradigms:
 - **State estimation fusion**: raw measurements combined at the signal level to estimate a shared physical state (e.g. Kalman filter, complementary filter).
 - **Decision making fusion**: each sensor independently extracts features and classifies objects; the high-level symbolic outputs are then combined (e.g. autonomous car perception pipeline).
- Sensors are most valuable when they are **complementary**: each sensor’s weaknesses are covered by the others’ strengths, and their noise sources are independent so errors do not compound.
- The image fusion examples illustrate both paradigms: focus fusion (same scene, reduce blind spots) mirrors the multi-sensor Kalman filter; MRI+CT fusion (different physical information) mirrors the complementary filter and IMU fusion in later sections.

Section 2 Kalman Filter for Sensor Fusion

We now turn to the state-estimation flavour of sensor fusion: multiple sensors all measuring aspects of the *same physical state*, combined inside a single Kalman filter. The good news is that the standard KF theory from earlier lectures carries over almost unchanged — the only new wrinkle is that several sensors may have *correlated* noise between them, which we need to handle before the standard update equations can be applied. This section develops that machinery step by step.

2.1 The Multi-Sensor Discrete-Time Model



► Discrete-time model with multiple sensor measurements:

$$\begin{aligned}
 x_{k+1} &= \Phi_k x_k + \Gamma_k u_k + w_k \\
 y_{1,k} &= H_{1,k} x_k + \nu_{1,k} \\
 y_{2,k} &= H_{2,k} x_k + \nu_{2,k} \\
 &\vdots \\
 y_{p_s,k} &= H_{p_s,k} x_k + \nu_{p_s,k}
 \end{aligned}$$

Figure 5: Block diagram for a discrete-time system with p_s sensors. The state x_k evolves through the matrix Φ_k (the state transition) with input u_k via gain Γ_k and is driven by process noise w_k . Each of the p_s sensors has its own observation matrix $H_{i,k}$ and adds its own measurement noise $\nu_{i,k}$ to produce output $y_{i,k}$. The z^{-1} block is the unit delay: it holds the state for one time step, representing the discrete-time integration. The diagram makes clear that all sensors observe the *same* state x_k — they differ only in which part of it they are sensitive to (which rows of $H_{i,k}$ are nonzero) and how noisy their channel is.

The system model is a straightforward extension of the single-sensor discrete-time KF model. The state evolves as before; the only change is that instead of one measurement equation there are now p_s of them — one per sensor:

$$x_{k+1} = \Phi_k x_k + \Gamma_k u_k + w_k$$

$$y_{1,k} = H_{1,k} x_k + \nu_{1,k}$$

$$y_{2,k} = H_{2,k} x_k + \nu_{2,k}$$

$$\vdots$$

$$y_{p_s,k} = H_{p_s,k} x_k + \nu_{p_s,k}$$

Symbol	Name	Meaning
x_k	State vector	The full physical state of the system at time step k (e.g. position, velocity, orientation).
Φ_k	State transition matrix	Propagates the state from step k to step $k+1$ under the system dynamics. If the dynamics are time-invariant, $\Phi_k = \Phi$ is constant.
Γ_k	Input matrix	Maps the known control input u_k into the state evolution.
w_k	Process noise	Models unmodelled dynamics and disturbances. Assumed $\mathcal{N}(0, Q_k)$.
p_s	Number of sensors	How many distinct sensors are fused. Each produces its own output $y_{i,k}$.
$y_{i,k}$	Output of sensor i	The raw measurement from sensor i at time k . May have different physical units to $y_{j,k}$ for $j \neq i$.
$H_{i,k}$	Observation matrix of sensor i	Maps the state to what sensor i actually measures. Different sensors can observe entirely different parts of the state.
$\nu_{i,k}$	Measurement noise of sensor i	The noise on sensor i 's reading. Its variance is $R_{ii,k}$; its correlation with sensor j is $R_{ij,k}$.

Why different $H_{i,k}$ matrices matter — concrete examples

Suppose the full state vector is position and velocity in 3D:

$$x_k = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} \quad \leftarrow \text{position (metres)} \quad \leftarrow \text{velocity (m/s)}$$

Three sensors are fused, each observing a different subset of this state.

Sensor 1 — GPS (measures full 3D position):

GPS gives you (x, y, z) directly. It says nothing about velocity. So the measurement

equation is:

$$y_{\text{GPS}} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}}_{H_{\text{GPS}}} \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} + \nu_{\text{GPS}}$$

The first three columns of H_{GPS} are an identity block because GPS reads position states directly — a 1 in column i means “pick out state component i ”. The last three columns are all zero because GPS tells us *nothing* about velocity — putting a 0 there instructs the filter not to try to infer velocity from this sensor’s reading.

Sensor 2 — Barometer (measures altitude only):

A barometer measures only vertical position z — one number. It knows nothing about x , y , or any velocity component:

$$y_{\text{baro}} = [z] = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}}_{H_{\text{baro}}} \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} + \nu_{\text{baro}}$$

Column 3 is 1 (pick out z); every other column is 0 (ignore everything else). The 0 in column 1 means: “this sensor gives us no information about x , so do not let it influence the x estimate.” The 0 in column 4 means: “this sensor gives us no information about v_x , so do not let it influence the velocity estimate.” The filter will use the barometer *only* to correct its z estimate, leaving all other states unchanged by this particular measurement.

Sensor 3 — Doppler radar (measures radial velocity only):

A Doppler radar measures how fast the vehicle is approaching or receding — that is, the velocity component along the beam direction. If the beam points along the x -axis, it measures v_x only:

$$y_{\text{Doppler}} = [v_x] = \underbrace{\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}}_{H_{\text{Doppler}}} \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix} + \nu_{\text{Doppler}}$$

Now the 1 appears in column 4 (the v_x component) and all position columns are 0. The Doppler radar directly constrains the velocity estimate without touching the position estimate at all.

All three fused in one filter:

When all three sensors are active at the same step, the stacked measurement vector is:

$$y_k = \begin{bmatrix} y_{\text{GPS}} \\ y_{\text{baro}} \\ y_{\text{Doppler}} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}}_{H_k} x_k + \nu_k$$

Notice that the z state appears in *both* row 3 (GPS) and row 4 (barometer) — the filter automatically exploits both measurements to get a better z estimate than either sensor could provide alone. The velocity state v_x appears only in row 5 (Doppler), so only that sensor drives the velocity estimate. States v_y and v_z have no 1 in any row — they are entirely unobserved by this sensor set and can only be tracked via the dynamic model.

What “tracked via the dynamic model” means — the Φ_k matrix.

Yes, the dynamic model here is the state transition equation governed by Φ_k . Even though no sensor ever directly reads v_y , the filter still maintains an estimate of it because Φ_k *couples* v_y to the position y , which GPS *does* observe.

In a constant-velocity model the state transition is:

$$\begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}_{k+1} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 & T_s & 0 \\ 0 & 0 & 1 & 0 & 0 & T_s \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\Phi_k} \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}_k$$

Row 2 of Φ_k says $y_{k+1} = y_k + T_s v_{y,k}$. GPS measures y at every step. So the filter sees that y changed by some amount between steps, and because it knows Φ_k predicts $\Delta y = T_s v_y$, it can *infer* what v_y must have been — even though v_y was never directly measured. Over several GPS updates, the estimate of v_y converges to the true value through this indirect coupling alone.

In short: H_k tells the filter what is *directly* observed. Φ_k tells the filter how the unobserved states are *linked* to the observed ones through the physics. Together they make it possible to estimate the full state even when no single sensor sees all of it.

2.2 Assumptions — The Correlated Noise Matrix R_k

- Initial state and covariance estimates are $x_0 \in \mathcal{N}(\hat{x}_{0|-1}, P_{0|-1})$, as usual.
- The process noise is – also as usual – assumed to be Gaussian and independently distributed:

$$w_k \in \mathcal{N}(0, Q_k)$$

- The *measurement* noise is assumed to be white for each sensor, but not necessarily uncorrelated between individual sensors:

$$\nu_k \in \mathcal{N}(0, R_k), \quad R_k = \begin{bmatrix} R_{11,k} & R_{12,k} & \cdots & R_{1p_s,k} \\ R_{12,k} & R_{22,k} & \cdots & R_{2p_s,k} \\ \vdots & \vdots & \ddots & \vdots \\ R_{p_s 1,k} & R_{p_s 2,k} & \cdots & R_{p_s p_s,k} \end{bmatrix}.$$

Figure 6: Kalman filter assumptions for the multi-sensor case. The process noise assumption (Gaussian, independent) is identical to the single-sensor case. The new element is the *measurement noise covariance matrix* R_k , which is now a $p_s \times p_s$ block matrix. The diagonal blocks $R_{ii,k}$ are the noise variances of the individual sensors; the off-diagonal blocks $R_{ij,k}$ ($i \neq j$) capture the *correlations* between the noise of sensor i and sensor j .

The standard KF assumptions carry over:

- The initial state is Gaussian: $x_0 \sim \mathcal{N}(\hat{x}_{0|-1}, P_{0|-1})$.
- The process noise is Gaussian and independent across time steps: $w_k \sim \mathcal{N}(0, Q_k)$.

The one new assumption concerns the **joint measurement noise**. Stack all p_s sensor noise vectors into one:

$$\nu_k = \begin{bmatrix} \nu_{1,k} \\ \nu_{2,k} \\ \vdots \\ \nu_{p_s,k} \end{bmatrix} \sim \mathcal{N}(0, R_k), \quad R_k = \begin{bmatrix} R_{11,k} & R_{12,k} & \cdots & R_{1p_s,k} \\ R_{12,k} & R_{22,k} & \cdots & R_{2p_s,k} \\ \vdots & \vdots & \ddots & \vdots \\ R_{p_s 1,k} & R_{p_s 2,k} & \cdots & R_{p_s p_s,k} \end{bmatrix}$$

Each sensor's noise is assumed white (uncorrelated with its own past values), but the noise of different sensors *at the same time step* may be correlated with each other.

What each entry of R_k means

Think of R_k as a symmetric matrix that fully describes the noise behaviour of all sensors at time step k :

- **Diagonal entry** $R_{ii,k}$: the *variance* of the noise on sensor i alone. Large $R_{ii,k}$ means sensor i is noisy and should be trusted less by the filter. Small $R_{ii,k}$ means sensor i is accurate and should be trusted more. This is exactly the R from the single-sensor KF.
- **Off-diagonal entry** $R_{ij,k}$ ($i \neq j$): the *covariance* between the noise on sensor i and the noise on sensor j at the same time instant. If $R_{ij,k} > 0$, sensors i and j tend to make their errors in the same direction at the same time. If $R_{ij,k} = 0$, the two sensors' errors are independent of each other.

When do off-diagonal terms arise in practice? They appear when two sensors share a common noise source:

- Two accelerometers on the same rigid body both see the same vibration noise from the structure they are mounted on.
- Two GPS antennas at the same location receive the same atmospheric delay from the same satellites.
- Two temperature sensors in the same enclosure are both affected by the same thermal environment.

In all these cases, when one sensor's error spikes, the other's tends to spike in the same direction — and the off-diagonal term captures exactly that tendency.

Why you cannot simply ignore the off-diagonal terms

If you incorrectly assume R_k is diagonal when it is not (i.e. you set all $R_{ij} = 0$ for $i \neq j$), the Kalman filter still runs — but it will produce a *suboptimal* estimate.

The filter weights each sensor independently, not knowing that sensors i and j are carrying redundant information. It may over-weight evidence that appears to come from two independent sources but is actually one shared error appearing twice. The estimated covariance $P_{k|k}$ will be too optimistic (too small), because the filter thinks it has more independent information than it actually does.

Correctly accounting for R_{ij} — which is what the Cholesky decorrelation in Section 2.3 achieves — ensures the filter is truly optimal.

2.3 Eliminating Correlations via Cholesky Factorisation

- If we know R_k , it is often convenient to change measurement coordinates such that the cross-correlations are eliminated.

- Factorize R_k as

$$R_k = L_k^\top L_k$$

with L_k being lower triangular (Cholesky factorization).

- Let $T_k = (L_k^{-1})^\top$ and stack the output equations:

$$y_k = \begin{bmatrix} y_{1,k} \\ \vdots \\ y_{n_p,k} \end{bmatrix}, \quad H_k = \begin{bmatrix} H_{1,k} \\ \vdots \\ H_{n_p,k} \end{bmatrix}, \quad \nu_k = \begin{bmatrix} \nu_{1,k} \\ \vdots \\ \nu_{n_p,k} \end{bmatrix}.$$

- Define new outputs

$$\bar{y}_k = T_k y_k = T_k (H_k x_k + \nu_k) = \bar{H}_k x_k + \bar{\nu}_k$$

where

$$\bar{\nu}_k \in \mathcal{N}(0, \bar{R}_k), \quad \bar{R}_k = T_k R_k T_k^\top = (L_k^{-1})^\top L_k^\top L_k L_k^{-1} = I.$$

Figure 7: The decorrelation procedure. **Top:** R_k is factorised as $R_k = L_k^\top L_k$ (Cholesky). **Middle:** the stacked output vector y_k , stacked observation matrix H_k , and stacked noise vector ν_k are defined. **Bottom:** new transformed outputs $\bar{y}_k = T_k y_k$ are defined, where $T_k = (L_k^{-1})^\top$. The transformed noise $\bar{\nu}_k$ has covariance $\bar{R}_k = I$ — the sensors are now uncorrelated *and* normalised to unit variance, making the standard KF update directly applicable.

The correlated R_k prevents us from directly plugging the stacked measurements into the standard Kalman filter, which assumes a diagonal (or at least known, manageable) R . The solution is to find a linear transformation T_k that **simultaneously decorrelates all sensor channels** and rescales them to unit variance. After the transformation, the standard KF update applies without any modification.

Step 1 — Stack all sensor equations into one.

Define the stacked vectors and matrices:

$$y_k = \begin{bmatrix} y_{1,k} \\ \vdots \\ y_{p_s,k} \end{bmatrix}, \quad H_k = \begin{bmatrix} H_{1,k} \\ \vdots \\ H_{p_s,k} \end{bmatrix}, \quad \nu_k = \begin{bmatrix} \nu_{1,k} \\ \vdots \\ \nu_{p_s,k} \end{bmatrix}$$

so that all p_s measurement equations collapse into one:

$$y_k = H_k x_k + \nu_k, \quad \nu_k \sim \mathcal{N}(0, R_k)$$

This is now formally identical to the single-sensor KF model, except that R_k is not diagonal.

Step 2 — Cholesky factorisation of R_k .

Since R_k is a covariance matrix it is symmetric positive definite. Every such matrix has a unique **Cholesky factorisation**:

$$R_k = L_k^\top L_k$$

where L_k is a **lower triangular matrix** with positive diagonal entries.

What Cholesky factorisation is and why it works here

The Cholesky decomposition is the matrix analogue of taking the square root of a positive number. Just as any positive number r can be written as $r = \sqrt{r} \cdot \sqrt{r}$, any symmetric positive definite matrix R_k can be written as $R_k = L_k^\top L_k$, where L_k plays the role of $\sqrt{R_k}$.

The lower triangular structure of L_k is not just a convenience — it is what makes the factorisation unique and numerically efficient. A $p_s \times p_s$ matrix has p_s^2 entries, but a lower triangular one has only $p_s(p_s + 1)/2$ entries. The Cholesky algorithm computes these entries in a single downward pass through the matrix, column by column. It is numerically stable and is the standard method used whenever a covariance matrix needs to be “square-rooted”.

Why write $R_k = L_k^\top L_k$ and not $R_k = L_k L_k^\top$? Both factorisations exist. The convention $R_k = L_k^\top L_k$ (with the transpose on the left) is chosen here because it makes $T_k = (L_k^{-1})^\top$ come out cleanly in Step 3. In MATLAB, `chol(R)` returns $L_k^\top$ by default (an upper triangular matrix), so the convention matches directly.

Step 3 — Define the decorrelating transformation T_k .

Let:

$$T_k = (L_k^{-1})^\top$$

This is the inverse of $L_k^\top$ — equivalently, it is the transpose of the inverse of L_k . Now multiply both sides of the stacked measurement equation by T_k :

$$\underbrace{T_k y_k}_{\bar{y}_k} = T_k H_k x_k + T_k \nu_k = \underbrace{T_k H_k}_{\bar{H}_k} x_k + \underbrace{T_k \nu_k}_{\bar{\nu}_k}$$

defining the **transformed output** $\bar{y}_k$, **transformed observation matrix** $\bar{H}_k = T_k H_k$, and **transformed noise** $\bar{\nu}_k = T_k \nu_k$.

Step 4 — Show that the transformed noise has identity covariance.

Compute the covariance of $\bar{\nu}_k$:

$$\bar{R}_k = \text{Cov}(\bar{\nu}_k) = T_k R_k T_k^\top = (L_k^{-1})^\top \cdot L_k^\top L_k \cdot ((L_k^{-1})^\top)^\top$$

Simplify the rightmost factor: $((L_k^{-1})^\top)^\top = L_k^{-1}$, so:

$$\bar{R}_k = (L_k^{-1})^\top L_k^\top \cdot L_k L_k^{-1} = (L_k L_k^{-1})^\top \cdot (L_k L_k^{-1}) = I^\top \cdot I = I$$

The transformed noise has **identity covariance**: $\bar{\nu}_k \sim \mathcal{N}(0, I)$. Every channel is uncorrelated with every other and has unit variance.

What the transformation actually does to the measurements — intuition

Before the transformation, the p_s sensor channels are *entangled*: sensors i and j share part of the same error, so the two readings are not truly independent pieces of information. Using both naively would count the shared error twice.

The matrix $T_k = (L_k^{-1})^\top$ *untangles* them. Think of it as rotating and rescaling the measurement space so that the cross-sensor correlations disappear. After the transformation:

- The channels of $\bar{y}_k$ are *independent* (zero off-diagonal covariance in $\bar{R}_k = I$).
- Each channel has *unit variance* (diagonal entries of $\bar{R}_k$ are all 1).

This means the transformed measurements $\bar{y}_k$ look exactly like independent, equally-weighted sensor readings — which is exactly what the standard KF update assumes. No information has been lost or distorted: the transformation is invertible (multiply by T_k^{-1} to recover the original), so it is a lossless change of coordinate system for the measurements.

Why normalising to unit variance matters: After decorrelation, each transformed channel contributes equally to the Kalman gain computation. The filter’s job then reduces to optimally weighting how informative each channel is about the state — which it does via the updated $\hat{H}_k$ and the predicted state covariance $P_{k|k-1}$.

Where does R_k come from in practice?

R_k is not computed by the filter — you must supply it. It describes the noise statistics of your sensors and comes from two sources:

- **Diagonal entries $R_{ii,k}$ — individual sensor noise.** These are the easiest to get. Point each sensor at a known, fixed value and record a long sequence of readings. The sample variance of those readings gives you R_{ii} . Many sensor datasheets also quote a noise figure (e.g. “RMS noise = 0.05 m”), from which $R_{ii} = 0.05^2 = 0.0025 \text{ m}^2$.
- **Off-diagonal entries $R_{ij,k}$ — cross-sensor correlations.** These are harder. You must run both sensors simultaneously, again pointed at a fixed known value, and compute the sample cross-covariance of their error sequences. If you have N simultaneous readings:

$$\hat{R}_{ij} = \frac{1}{N} \sum_{t=1}^N (y_{i,t} - \bar{y}_{i,t})(y_{j,t} - \bar{y}_{j,t})$$

where $\bar{y}_{i,t}$ is the true value sensor i should have read. If the sensors are physically separate and driven by different electronics, R_{ij} is often zero or negligible and you can safely set it to zero. It becomes significant when two sensors share a power supply, a common clock, or are affected by the same environmental disturbance.

What do you actually apply T_k to? — A complete worked example

T_k must be applied to **two things**: the observation matrix H_k and the actual measurement y_k at every step. Nothing else changes — the state equations, $P_{k|k-1}$, and Q_k are all untouched.

Setup. Two sensors both measure the same scalar state x_k (e.g. altitude), so both observation matrices are simply 1:

$$H_{1,k} = H_{2,k} = 1 \implies H_k = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Sensor 1 is noisier (variance 4), sensor 2 is more accurate (variance 2), and they share a common error source giving covariance 2:

$$R_k = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$$

Step 1 — Cholesky factorisation $R_k = L_k^\top L_k$.

Requirement for Cholesky to exist: R_k must be **symmetric positive definite** (SPD). Since R_k is a covariance matrix of real sensor noise it is always symmetric, and positive definiteness is guaranteed as long as no sensor channel is a perfect deterministic function of another. If R_k is only positive *semidefinite* (a degenerate case), Cholesky fails and the sensors must be reduced to a linearly independent subset first.

Find lower-triangular L_k such that $L_k^\top L_k = R_k$:

$$L_k = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$$

Verify:

$$L_k^\top L_k = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} = R_k \checkmark$$

Step 2 — Compute $T_k = (L_k^{-1})^\top$.

First invert L_k . For a 2×2 lower triangular matrix this is straightforward:

$$L_k^{-1} = \begin{bmatrix} 1/2 & 0 \\ -1/2 & 1 \end{bmatrix}$$

$$\text{Verify: } L_k^{-1} L_k = \begin{bmatrix} 1/2 & 0 \\ -1/2 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \checkmark$$

Now transpose:

$$T_k = (L_k^{-1})^\top = \begin{bmatrix} 1/2 & -1/2 \\ 0 & 1 \end{bmatrix}$$

Step 3 — Apply T_k to H_k : get $\bar{H}_k$.

$$\bar{H}_k = T_k H_k = \begin{bmatrix} 1/2 & -1/2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Interesting: the first transformed channel has $\bar{H}_1 = 0$ — it carries no information about x_k at all after the transformation. The second transformed channel has $\bar{H}_2 = 1$ — it is the only informative channel.

Why did sensor 1 become completely useless? Yes — in this example sensor 1 was essentially just a noisier, worse version of sensor 2. Think about what the two sensors gave us:

- Sensor 2: reads the true state plus some noise ν_2 (variance 2).
- Sensor 1: reads the same true state plus the *same* noise ν_2 (the shared part, covariance 2) *and* an extra independent noise term on top (bringing its total variance up to 4).

So sensor 1 = sensor 2's reading + extra garbage. Once you already have sensor 2, sensor 1 tells you nothing new — everything it contains is either already in sensor 2 or is additional noise you do not want. The transformation discovers this automatically: the linear combination $\frac{1}{2}y_1 - \frac{1}{2}y_2$ (the first row of T_k) cancels out the true state entirely, leaving only noise. Naturally $\bar{H}_1 = 0$ — the filter simply discards it.

This is not always what happens. It is a consequence of the correlation being very strong here ($R_{12} = 2$ out of a theoretical maximum of $\sqrt{R_{11} \cdot R_{22}} = \sqrt{4 \cdot 2} \approx 2.83$, so about 71% of the maximum possible correlation). With weaker correlation both transformed channels would still have nonzero $\bar{H}$ values and both would contribute to the update, just with appropriately adjusted weights. The transformation does not decide to throw sensors away — it simply exposes the truth about how much independent information each sensor actually carries, and in this case sensor 1 carried none.

Step 4 — Apply T_k to the actual measurement y_k : get $\bar{y}_k$.

Suppose at this time step sensor 1 reads $y_{1,k} = 10.3$ m and sensor 2 reads $y_{2,k} = 9.8$ m:

$$\bar{y}_k = T_k y_k = \begin{bmatrix} 1/2 & -1/2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 10.3 \\ 9.8 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 9.8 \end{bmatrix}$$

The first transformed reading is 0.25 but its $\bar{H}_1 = 0$, so it contributes nothing to the state update — the filter ignores it. The second transformed reading 9.8 has $\bar{H}_2 = 1$ and this is what the filter uses to update its altitude estimate.

Step 5 — Verify $\bar{R}_k = I$.

$$\bar{R}_k = T_k R_k T_k^\top = \begin{bmatrix} 1/2 & -1/2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 1/2 & 0 \\ -1/2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I \checkmark$$

The two transformed channels now have unit variance and zero cross-correlation — their noise is completely independent. The standard KF update can be applied directly using $\bar{H}_k$ and $\bar{R}_k = I$.

Summary of what T_k is applied to:

Quantity	Before transformation	After transformation
Observation matrix	H_k	$\bar{H}_k = T_k H_k$
Actual measurement	y_k	$\bar{y}_k = T_k y_k$
Noise covariance	R_k (correlated)	$\bar{R}_k = I$ (independent)
State / covariance	$x_k, P_{k k-1}$	unchanged

2.4 The Full KF Update with Transformed Measurements

- Initial conditions: $x_0 \in \mathcal{N}(x_{0|-1}, P_{0|-1})$.
- Measurement update¹ after receiving y_k and u_k :

$$\begin{aligned}
T_k &= \text{chol}(R_k) \\
\hat{\bar{y}}_{k|k-1} &= \bar{H}_k \hat{x}_{k|k-1} \\
\tilde{y}_{k|k-1} &= T_k y_k - \hat{\bar{y}}_{k|k-1} \\
K_k &= P_{k|k-1} \bar{H}_k^\top (\bar{H}_k^\top P_{k|k-1} \bar{H}_k + \bar{R}_k)^{-1} \\
\hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k \tilde{y}_{k|k-1} \\
P_{k|k} &= (I - K_k \bar{H}_k)^{-1} P_{k|k-1} (I - K_k \bar{H}_k)^\top + K_k \bar{R}_k K_k^\top.
\end{aligned}$$

- Time update from k to $k+1$:

$$\begin{aligned}
\hat{x}_{k+1|k} &= \Phi_k \hat{x}_{k|k} + \Gamma_k u_k \\
P_{k+1|k} &= \Phi_k P_k \Phi_k^\top + Q_k
\end{aligned}$$

¹Obs! Note that Yan et. al. use a different formulation for the $P_{k|k}$ update, namely

$$P_{k|k} = (I - K_k \bar{H}_k) P_{k|k-1}$$

The two expressions are, in fact, identical, although it takes a healthy dose of algebra to show it.

Figure 8: Complete Kalman filter equations for multi-sensor fusion using the transformed measurements. The footnote on the slide notes that an alternative (Joseph) form of the $P_{k|k}$ update is used here, which is numerically more stable than the simple $(I - K_k \bar{H}_k) P_{k|k-1}$ form; the two are algebraically identical.

Once the transformation T_k has been computed and applied, the Kalman filter runs in exactly the same form as the single-sensor case, but on the transformed quantities $(\bar{y}_k, \bar{H}_k, \bar{R}_k = I)$.

Initialisation:

$$x_0 \sim \mathcal{N}(\hat{x}_{0|-1}, P_{0|-1})$$

Measurement update (after receiving y_k and u_k at step k):

First compute the transformation:

$$T_k = \text{chol}(R_k)$$

What chol returns — matching the convention

In MATLAB, `chol(R)` returns the upper triangular factor U such that $R = U^\top U$. Comparing with our convention $R_k = L_k^\top L_k$, we have $U = L_k$ and so `chol(R)` gives L_k directly. Therefore $T_k = (L_k^{-1})^\top = (\text{chol}(R)^{-1})^\top$. In the slide the notation $T_k = \text{chol}(R_k)$ is shorthand for this full computation.

Predicted (transformed) output:

$$\hat{y}_{k|k-1} = \bar{H}_k \hat{x}_{k|k-1}$$

Transformed innovation (the “surprise” from the new measurements):

$$\tilde{y}_{k|k-1} = T_k y_k - \hat{y}_{k|k-1}$$

Kalman gain:

$$K_k = P_{k|k-1} \bar{H}_k^\top (\bar{H}_k P_{k|k-1} \bar{H}_k^\top + \bar{R}_k)^{-1}$$

State estimate update:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_{k|k-1}$$

Covariance update (Joseph/symmetric form):

$$P_{k|k} = (I - K_k \bar{H}_k) P_{k|k-1} (I - K_k \bar{H}_k)^\top + K_k \bar{R}_k K_k^\top$$

Time update (propagation from step k to $k+1$):

$$\hat{x}_{k+1|k} = \Phi_k \hat{x}_{k|k} + \Gamma_k u_k$$

$$P_{k+1|k} = \Phi_k P_{k|k} \Phi_k^\top + Q_k$$

Symbol	Name	Meaning
T_k	Decorrelating transform	$T_k = (L_k^{-1})^\top$ where L_k comes from Cholesky of R_k . Applied once per step before everything else.
$\bar{H}_k$	Transformed observation matrix	$\bar{H}_k = T_k H_k$. Used in place of H_k throughout the update step.
$\bar{R}_k$	Transformed noise covariance	$\bar{R}_k = I$ always, by construction of T_k . The standard KF sees perfectly decorrelated unit-variance channels.
$\hat{x}_{k k-1}$	Predicted state	Best estimate of x_k before the new measurements are used, carried over from the previous time update.
$\tilde{y}_{k k-1}$	Innovation	The gap between what was measured ($T_k y_k$, i.e. the transformed actual reading) and what was predicted ($\hat{y}_{k k-1}$). A large innovation means the model was surprised — either the measurement is informative, or the model is wrong.
K_k	Kalman gain	Weights how much the innovation corrects the state estimate. Larger K_k means more trust in the measurements; smaller means more trust in the model.
$P_{k k}$	Updated state covariance	How uncertain the state estimate is after incorporating the measurements. Should be smaller than $P_{k k-1}$.
$P_{k+1 k}$	Predicted state covariance	Propagated uncertainty before the next measurement update. Grows due to process noise Q_k .

Why the Joseph form of the covariance update is used

The standard simplified covariance update is:

$$P_{k|k} = (I - K_k \bar{H}_k) P_{k|k-1}$$

This is algebraically correct *only when* K_k is exactly the optimal Kalman gain. In finite-precision computer arithmetic, rounding errors accumulate and K_k is never computed perfectly. Even small errors in K_k can cause the simplified formula to produce a $P_{k|k}$ that is slightly non-symmetric or even not positive semi-definite — which would cause the filter to diverge or produce nonsensical negative variances.

The **Joseph (symmetric) form**:

$$P_{k|k} = (I - K_k \bar{H}_k) P_{k|k-1} (I - K_k \bar{H}_k)^\top + K_k \bar{R}_k K_k^\top$$

is *always* symmetric and positive semi-definite by construction, even with non-optimal K_k and finite-precision arithmetic. The extra $K_k \bar{R}_k K_k^\top$ term adds back the measure-

ment noise contribution that is lost when K_k is not exactly optimal, ensuring the covariance is never underestimated. This is the numerically safe form and should always be preferred in implementation.

The full algorithm at a glance — what runs each step

At every time step k the filter does the following, in order:

1. **Receive** the new measurement vector y_k (all sensors stacked) and the known input u_k .
2. **Compute** $T_k = \text{chol}(R_k)$ and $\bar{H}_k = T_k H_k$. If R_k is constant over time this is done once offline.
3. **Form the innovation:** $\tilde{y}_{k|k-1} = T_k y_k - \bar{H}_k \hat{x}_{k|k-1}$.
4. **Compute the Kalman gain:** $K_k = P_{k|k-1} \bar{H}_k^\top (\bar{H}_k P_{k|k-1} \bar{H}_k^\top + I)^{-1}$. Note: $\bar{R}_k = I$ so no R appears in the gain formula.
5. **Update state and covariance:** $\hat{x}_{k|k}$ and $P_{k|k}$ via the formulas above.
6. **Time update:** propagate $\hat{x}_{k+1|k} = \Phi_k \hat{x}_{k|k} + \Gamma_k u_k$ and $P_{k+1|k} = \Phi_k P_{k|k} \Phi_k^\top + Q_k$.

Steps 2–5 are the *measurement update*; step 6 is the *time update*. The two-step structure is identical to the single-sensor KF — the Cholesky transformation in step 2 is the only addition. If R_k is constant (time-invariant sensor noise), T_k and $\bar{H}_k$ are precomputed once and the runtime cost is exactly the same as a standard KF.

Units — the most common practical mistake in multi-sensor fusion

The slide states that different sensors can give measurements “in vastly different units.” This is not just a cosmetic issue — it directly affects the filter.

Suppose sensor 1 measures position in metres ($y_1 \sim 1\text{--}100\text{ m}$) and sensor 2 measures it in millimetres ($y_2 \sim 1000\text{--}100,000\text{ mm}$). The raw numerical values differ by a factor of 1000. If you stack them directly without converting to common units, the R_k matrix will have entries that differ by 10^6 (the square of 1000), and the Kalman gain will be wildly skewed toward the sensor with the numerically larger readings — which is the less precise one in physical terms.

Rule: always convert all sensor outputs to a consistent set of units (e.g. SI: metres, radians, seconds) before stacking them into y_k and forming R_k . The Cholesky decorrelation does not fix unit mismatches — it only removes statistical correlations between correctly-scaled channels.

Section 2 — Key Takeaways

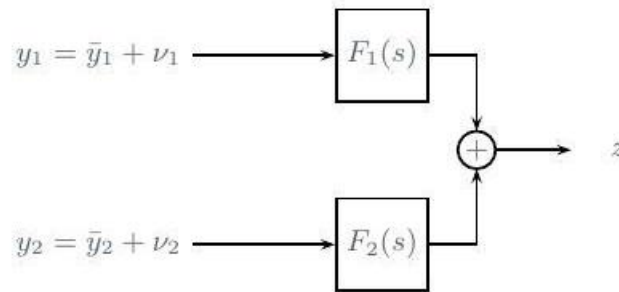
- The multi-sensor model has one state equation and p_s measurement equations, one per sensor: $y_{i,k} = H_{i,k} x_k + \nu_{i,k}$. Different $H_{i,k}$ matrices allow sensors measuring completely different physical quantities to be fused inside one filter.

- The joint noise covariance R_k is a $p_s \times p_s$ matrix. **Diagonal entries** $R_{ii,k}$ are individual sensor variances; **off-diagonal entries** $R_{ij,k}$ capture noise correlations between sensors sharing a common error source. Ignoring off-diagonal terms gives a suboptimal, overconfident filter.
- **Cholesky decorrelation:** factorise $R_k = L_k^\top L_k$, set $T_k = (L_k^{-1})^\top$, and transform: $\bar{y}_k = T_k y_k$, $\bar{H}_k = T_k H_k$, $\bar{R}_k = T_k R_k T_k^\top = I$. The transformed channels are independent and unit-variance.
- After the transformation the **standard KF update** applies unchanged, with $\bar{H}_k$ and $\bar{R}_k = I$ in place of H_k and R_k . Use the **Joseph form** of the covariance update to maintain positive definiteness under finite-precision arithmetic.
- Always convert all sensor readings to **consistent units** before stacking — unit mismatches corrupt R_k and bias the Kalman gain.

Section 3 Complementary Filter

The Kalman filter from Section 2 is the optimal solution for multi-sensor fusion when you have a state-space model, Gaussian noise, and known noise covariances. But sometimes the situation is simpler: you have *two sensors that measure related quantities* — one good at low frequencies, one good at high frequencies — and you want a clean estimate of the underlying signal without building a full state-space model. The **complementary filter** is the tool for this. It is simple, computationally cheap, and widely used in embedded systems such as drones and robots where the full KF overhead is not justified.

3.1 Core Idea — Splitting the Frequency Spectrum



- ▶ $F_1(s), F_2(s)$: filters
- ▶ $z = F_1(s)y_1 + F_2(s)y_2$: sum of filtered versions of both measurements.
- ▶ Complementary filter: $F_1(s) = I - F_2(s)$.
- ▶ Allows separation of filtering in different frequency ranges.

Figure 9: General complementary filter architecture. Two sensors produce noisy measurements y_1 and y_2 of related quantities. Each passes through its own filter $F_1(s)$ and $F_2(s)$. The outputs are summed to give the final estimate z . The complementary condition $F_1(s) = I - F_2(s)$ ensures the two filters together cover the full frequency spectrum without overlap or gap — whatever F_2 does not pass, F_1 picks up, and vice versa.

The general structure is:

$$z = F_1(s) y_1 + F_2(s) y_2$$

where y_1 and y_2 are two measurements, $F_1(s)$ and $F_2(s)$ are linear filters (transfer functions in the Laplace domain), and z is the fused estimate of the quantity of interest.

The defining property of a complementary filter is:

$$\boxed{F_1(s) = I - F_2(s)} \iff F_1(s) + F_2(s) = I$$

What $F_1(s) + F_2(s) = I$ means and why it is called “complementary”

Think of $F_2(s)$ as a **low-pass filter** — it passes slow, low-frequency content and blocks fast variations. Then $F_1(s) = I - F_2(s)$ is its **high-pass complement** — it passes exactly what the low-pass blocks and blocks what the low-pass passes.

Together, at every frequency:

- At **low frequencies**: $F_2 \approx 1$, so $F_1 \approx 0$. The output $z \approx y_2$ — the estimate is dominated by sensor 2.
- At **high frequencies**: $F_2 \approx 0$, so $F_1 \approx 1$. The output $z \approx y_1$ — the estimate is dominated by sensor 1.
- At **all frequencies combined**: $F_1 + F_2 = 1$, so no frequency is either double-counted or missing from the output.

This is where the name comes from: F_1 and F_2 are *complementary* in the literal sense — each covers the frequency range the other does not. The design question reduces to: choose F_2 (the low-pass), and F_1 is determined automatically.

What if both sensors capture some of the same frequency range?

If the two sensors overlap — both are reliable in some shared band — the complementary filter still runs without breaking, but it becomes **suboptimal** and you lose the main benefit.

Here is what happens in the overlap region. Because $F_1 + F_2 = 1$, the filter assigns every frequency either mostly to sensor 1 or mostly to sensor 2, depending on which side of the crossover $1/\tau$ that frequency sits. In the overlap band, where *both* sensors are reliable, the filter still only uses one of them at a time — it never combines both. You are therefore throwing away valid information from whichever sensor the filter is downweighting at that frequency.

A **Kalman filter** handles this correctly: it has a separate noise variance for each sensor and computes the optimal weighted average at every frequency, automatically giving more weight to the better sensor at each frequency and using both when they are equally reliable. The complementary filter cannot do this because it has only one tuning parameter (τ) to describe all trade-offs.

The extreme case — two sensors measuring exactly the same thing: If both sensors observe the same quantity at all frequencies (e.g. two GPS receivers of the same type), the complementary filter reduces to: use sensor 2 at low frequencies and sensor 1 at high frequencies. You are still only ever using one sensor at a time and completely ignoring the other’s reading in its assigned band, even though both are equally valid. The right tool here is a KF (or simply averaging the two readings with inverse-variance weights), not a complementary filter.

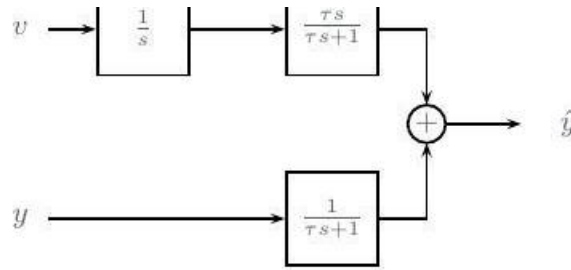
Rule of thumb: use a complementary filter when your sensors have clearly *non-overlapping* strength regions — one is good where the other is bad. When there is significant overlap, switch to a Kalman filter.

Why split by frequency? Different sensors tend to be reliable in different frequency ranges:

- A **gyroscope** integrates cleanly at high frequencies (fast rotations) but drifts at low frequencies (slow bias builds up over time).
- An **accelerometer or magnetometer** gives a stable low-frequency reference (gravity direction, magnetic north) but is corrupted by vibration at high frequencies.

Assigning the high-frequency band to the gyroscope and the low-frequency band to the accelerometer — and summing them with complementary filters — exploits the strength of each sensor while rejecting its weakness. This exact idea is at the heart of the Madgwick and Mahoney filters developed in Sections 6 and 7.

3.2 Continuous-Time Example: Position from Velocity and Position



- Suppose v and y are velocity and position measurements of some moving object. Ideally these measurements are related via

$$\bar{y} = \frac{1}{s}v.$$

- Let $F_2(s) = \frac{1}{\tau s + 1}$ be a low-pass filter. Then $F_1(s)$ becomes the high-pass filter

$$F_1(s) = 1 - \frac{1}{\tau s + 1} = \frac{\tau s}{\tau s + 1}.$$

- Position estimate:

$$z = \hat{y} = F_1(s)\frac{1}{s}v + F_2(s)y = \frac{1}{\tau s + 1}(\tau v + y).$$

Figure 10: Complementary filter for fusing a **velocity** measurement v and a **position** measurement y . The velocity is first integrated ($1/s$) to produce a position estimate, then passed through the high-pass filter $F_1(s) = \frac{\tau s}{\tau s + 1}$. The position measurement passes through the low-pass filter $F_2(s) = \frac{1}{\tau s + 1}$. Both are summed to give the fused position estimate $\hat{y}$. The time constant τ controls where the crossover between the two sensors occurs.

Suppose you have two sensors on a moving object:

- **Sensor 1:** measures *velocity* v (e.g. a tachometer or Doppler radar). Fast and accurate at high frequencies but cannot give you absolute position — you must integrate to get there.

- **Sensor 2:** measures *position* y directly (e.g. GPS). Gives absolute position but is slow and noisy at high frequencies.

If the sensor measurements were perfect, position and velocity would be related by:

$$\bar{y} = \frac{1}{s} v$$

where $1/s$ denotes integration in the Laplace domain ($\bar{y}$ is the ideal noise-free position, v is the noise-free velocity). In reality both measurements are noisy, and direct integration of a noisy velocity drifts without bound.

Choosing the filters.

Let $F_2(s)$ be a first-order low-pass filter with time constant τ :

$$F_2(s) = \frac{1}{\tau s + 1}$$

By the complementary condition, $F_1(s)$ is automatically:

$$F_1(s) = I - F_2(s) = 1 - \frac{1}{\tau s + 1} = \frac{\tau s + 1 - 1}{\tau s + 1} = \frac{\tau s}{\tau s + 1}$$

This is a first-order **high-pass filter** with the same time constant τ .

The fused position estimate.

Sensor 1 contributes via integration then high-pass filtering; sensor 2 contributes via direct low-pass filtering. The output is:

$$z = \hat{y} = F_1(s) \cdot \frac{1}{s} \cdot v + F_2(s) \cdot y = \frac{\tau s}{\tau s + 1} \cdot \frac{v}{s} + \frac{1}{\tau s + 1} \cdot y = \frac{\tau v}{\tau s + 1} + \frac{y}{\tau s + 1} = \frac{1}{\tau s + 1} (\tau v + y)$$

So the complete estimate is:

$$\hat{y} = \frac{1}{\tau s + 1} (\tau v + y)$$

Symbol	Name	Meaning
v	Velocity measurement	Raw reading from sensor 1 (e.g. tachometer, Doppler). Good at high frequencies; integrates to position but drifts.
y	Position measurement	Raw reading from sensor 2 (e.g. GPS). Good at low frequencies; gives absolute position but noisy at high frequencies.
$\hat{y}$	Fused position estimate	The complementary filter output — a weighted combination of both sensors that takes the best from each.
τ	Time constant	Sets the crossover frequency $\omega_c = 1/\tau$ between the two sensors. Below ω_c : GPS dominates. Above ω_c : velocity integration dominates.
$F_1(s) = \frac{\tau s}{\tau s + 1}$	High-pass filter	Applied to the integrated velocity. Passes fast position changes well; removes the low-frequency drift from integration.
$F_2(s) = \frac{1}{\tau s + 1}$	Low-pass filter	Applied to the GPS position. Passes the slow, absolute position reference; removes high-frequency GPS noise.

Intuition for the formula $\hat{y} = \frac{1}{\tau s + 1}(\tau v + y)$

Rewrite slightly by multiplying out:

$$(\tau s + 1)\hat{y} = \tau v + y$$

In the time domain this is a first-order differential equation:

$$\tau \dot{\hat{y}} + \hat{y} = \tau v + y$$

This is saying: the estimate $\hat{y}$ is driven by two terms:

- τv : the velocity measurement scaled by τ — this term drives $\hat{y}$ to change at the measured rate v . Since τv is proportional to velocity, it acts like integration of v (it adds the velocity contribution to the estimate).
- y : the GPS position — this term continuously pulls $\hat{y}$ back toward the absolute GPS reading, preventing drift.

The time constant τ controls the relative weight: large τ trusts velocity more (faster dynamics, more responsive to v); small τ trusts GPS more (estimate stays close to y).

What happens at the extremes of τ :

- $\tau \rightarrow 0$: $F_1 \rightarrow 0$, $F_2 \rightarrow 1$, $\hat{y} \rightarrow y$. The filter ignores the velocity entirely and just outputs the GPS reading directly.

- $\tau \rightarrow \infty$: $F_2 \rightarrow 0$, $F_1 \rightarrow 1$, $\hat{y} \rightarrow \frac{1}{s}v$ (pure velocity integration). The GPS is completely ignored and the estimate drifts freely.
- A finite τ : the GPS prevents long-term drift while the velocity integration gives smooth, high-rate position updates between slow GPS fixes.

How does this relate to the Kalman filter?

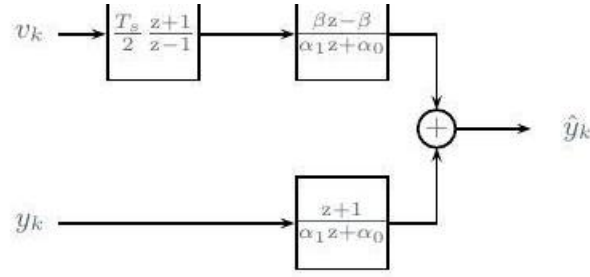
The complementary filter and the Kalman filter for the same problem (fusing position and velocity) produce *identical results* when the Kalman filter has converged and the noise statistics happen to match the implied weighting of the complementary filter.

The key difference is how τ is chosen:

- In the **complementary filter**, τ is a design parameter you set by engineering judgement — typically based on knowing roughly how noisy GPS is versus the velocity sensor.
- In the **Kalman filter**, the equivalent of τ emerges automatically from the noise covariances Q and R that you supply. The filter computes the optimal crossover frequency for you.

The complementary filter is therefore a computationally cheap, tuning-by-hand version of the Kalman filter for problems where the sensors split cleanly into “good at high frequencies” and “good at low frequencies”. When this split is clean, the complementary filter is often preferred in embedded systems for its simplicity and low CPU cost.

3.3 Discrete-Time Implementation



- Tustin approximation of integrator:

$$\bar{y}_k = \frac{T_s}{2} \frac{z+1}{z-1} v_k$$

- Low-pass filter: $F_2(z) = \frac{z+1}{\alpha_1 z + \alpha_0}$ where $\alpha_1 = \frac{\tau T_s}{2} + 1, \alpha_0 = 1 - \frac{\tau T_s}{2}$.
- High-pass complement: $F_1(z) = \frac{\beta(z-1)}{\alpha_1 z + \alpha_0}$ where $\beta = \alpha_1 - 1$.
- Position estimate:

$$\hat{y}_k = \frac{1}{\alpha_1} \left(-\alpha_0 \hat{y}_{k-1} + \beta \frac{T_s}{2} v_k + \beta \frac{T_s}{2} v_{k-1} + y_k + y_{k+1} \right)$$

Figure 11: Discrete-time complementary filter block diagram. The integrator $1/s$ is approximated by the Tustin (bilinear) method, giving the discrete-time factor $\frac{T_s}{2} \frac{z+1}{z-1}$. The low-pass filter F_2 and its high-pass complement F_1 are also expressed in the z -domain. All three blocks operate on sampled signals with sample period T_s , and their outputs are summed to give the discrete-time position estimate $\hat{y}_k$.

To implement the complementary filter on a digital processor, every continuous-time block must be converted to a discrete-time equivalent. The standard method is the **Tustin (bilinear) approximation**, which replaces the Laplace variable s with:

$$s \leftarrow \frac{2}{T_s} \cdot \frac{z-1}{z+1}$$

where T_s is the sample period and z is the z -transform variable (one step forward in time).

Why Tustin

- **Tustin / bilinear** ($s \leftarrow \frac{2}{T_s} \frac{z-1}{z+1}$): maps the left half of the s -plane *exactly* onto the interior of the unit disc, preserving stability. It also preserves the frequency response shape much more accurately than the Euler methods up to the Nyquist frequency.

For a complementary filter where precise frequency-domain behaviour matters (the crossover frequency $1/\tau$ must be where you designed it), Tustin is the right choice.

Tustin approximation of the integrator $1/s$:

$$\frac{1}{s} \longrightarrow \frac{T_s}{2} \cdot \frac{z+1}{z-1}$$

Applied to the velocity v_k , this gives the approximate integrated position:

$$\bar{y}_k = \frac{T_s}{2} \cdot \frac{z+1}{z-1} v_k$$

In the time domain: $\bar{y}_k = \bar{y}_{k-1} + \frac{T_s}{2}(v_k + v_{k-1})$ — this is simply the **trapezoidal integration rule**: the area under the velocity curve between steps $k-1$ and k is approximated as the average of the two endpoint velocities times the step width T_s .

Discrete-time low-pass filter F_2 :

Applying the Tustin substitution to $F_2(s) = \frac{1}{\tau s + 1}$:

$$F_2(z) = \frac{z+1}{\alpha_1 z + \alpha_0}$$

where the coefficients are:

$$\alpha_1 = \frac{\tau T_s}{2} + 1, \quad \alpha_0 = 1 - \frac{\tau T_s}{2}$$

Discrete-time high-pass complement F_1 :

Since $F_1(z) = 1 - F_2(z)$:

$$F_1(z) = 1 - \frac{z+1}{\alpha_1 z + \alpha_0} = \frac{(\alpha_1 z + \alpha_0) - (z+1)}{\alpha_1 z + \alpha_0} = \frac{(\alpha_1 - 1)z + (\alpha_0 - 1)}{\alpha_1 z + \alpha_0}$$

Define $\beta = \alpha_1 - 1 = \frac{\tau T_s}{2}$, and note that $\alpha_0 - 1 = -\frac{\tau T_s}{2} = -\beta$:

$$F_1(z) = \frac{\beta(z-1)}{\alpha_1 z + \alpha_0}$$

Symbol	Name	Meaning
T_s	Sample period	Time between successive measurements (seconds). Determines how fine the discrete approximation is.
τ	Time constant	Inherited from the continuous-time design. Sets the crossover frequency between the two sensors: $\omega_c = 1/\tau$ rad/s.
α_1	Filter coefficient	$\alpha_1 = \frac{\tau T_s}{2} + 1$. Always > 1 since $\tau, T_s > 0$.
α_0	Filter coefficient	$\alpha_0 = 1 - \frac{\tau T_s}{2}$. Positive as long as $T_s < 2\tau/1 = 2\tau$ — i.e. the sample rate is faster than the filter's time constant (always true in practice).
β	High-pass coefficient	$\beta = \alpha_1 - 1 = \frac{\tau T_s}{2}$. Controls the strength of the velocity contribution in F_1 .

The discrete-time position estimate.

Combining $F_1(z) \cdot \bar{y}$ (velocity integration through high-pass) and $F_2(z) \cdot y$ (position through low-pass), and converting from z -domain to a time-domain recursion (shifting by one step to make it causal), the implementable update equation is:

$$\hat{y}_k = \frac{1}{\alpha_1} \left(-\alpha_0 \hat{y}_{k-1} + \beta \frac{T_s}{2} v_k + \beta \frac{T_s}{2} v_{k-1} + y_k + y_{k-1} \right)$$

Reading the recursion term by term

At each time step k you compute the new position estimate from quantities that are all available right now or from the previous step — this is a fully causal, real-time implementable formula:

- $-\alpha_0 \hat{y}_{k-1}$: the **memory term** — the previous estimate, scaled and fed back. Because $\alpha_0 < \alpha_1$, this damps old estimates out exponentially, preventing unbounded growth.
- $\beta \frac{T_s}{2} v_k + \beta \frac{T_s}{2} v_{k-1}$: the **velocity integration term** — the trapezoidal integral of v over the current step, scaled by $\beta = \frac{\tau T_s}{2}$. This is the high-pass channel: it captures how much the position changed *since the last step* according to the velocity sensor.
- $y_k + y_{k-1}$: the **GPS correction term** — the average of the current and previous GPS readings. This is the low-pass channel: it continuously anchors the estimate to the absolute GPS position, preventing the velocity integral from drifting.
- $\frac{1}{\alpha_1}$: normalisation factor — divides everything by $\alpha_1 > 1$ so the output stays in the right scale.

Implementation in practice (e.g. on a microcontroller):

1. Read v_k (velocity sensor) and y_k (GPS/position sensor).
2. Compute $\hat{y}_k$ using the formula above, with v_{k-1} , y_{k-1} , $\hat{y}_{k-1}$ stored from last step.
3. Store $v_k \rightarrow v_{k-1}$, $y_k \rightarrow y_{k-1}$, $\hat{y}_k \rightarrow \hat{y}_{k-1}$ for the next step.
4. Output $\hat{y}_k$ as the fused position estimate.

That is the entire filter — four multiplications, four additions, and three stored values. No matrix inversions, no covariance propagation.

3.4 Filtering Performance

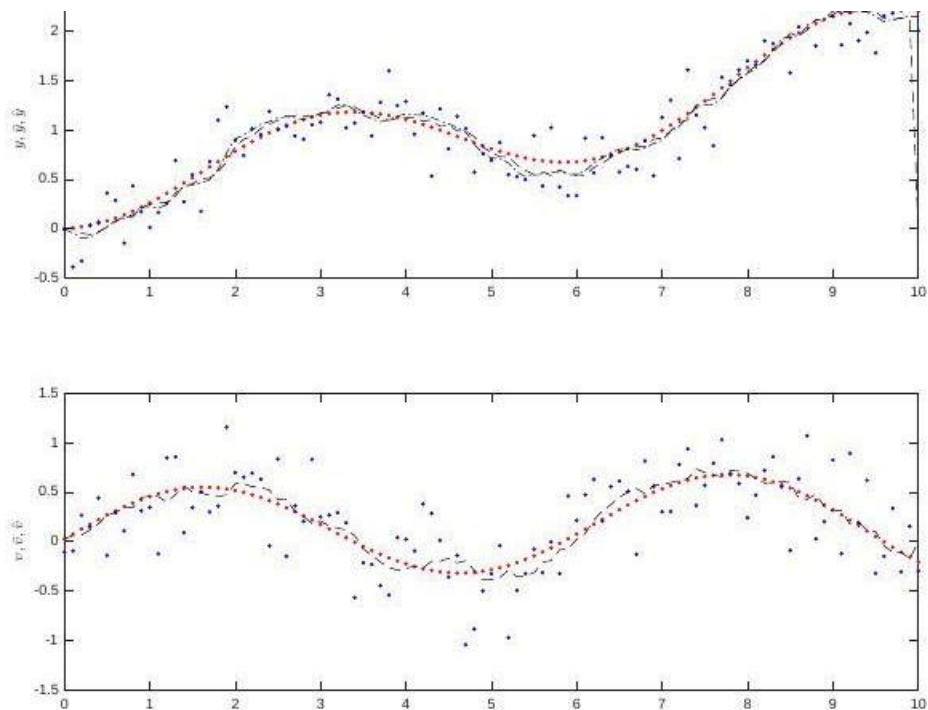


Figure 12: Complementary / Kalman filter performance on a simulated trajectory. **Red (dotted)**: the true deterministic signal. **Blue (dots)**: the noisy raw measurements. **Black (dashed)**: the filter estimate. **Top plot**: position y (or $\hat{y}$) vs. time. The measurements (blue) scatter widely around the true trajectory (red); the filter (black) tracks the true signal closely, smoothing out the noise while following the trend. **Bottom plot**: velocity v (or $\hat{v}$) vs. time. The velocity estimate from the filter (black) tracks the true velocity (red) while rejecting the measurement scatter. Neither position nor velocity is directly measured cleanly at every step — the filter reconstructs both from the combined information.

The performance plot shows the fundamental trade-off the time constant τ controls:

- **Too small τ** (trusts GPS too much): the estimate follows the raw GPS readings closely — it is noisy and jumpy, barely smoother than the blue dots. Every GPS error immediately corrupts the estimate.
- **Too large τ** (trusts velocity too much): the estimate integrates the velocity sensor for too long without GPS correction — it drifts away from the true position between GPS updates, and can be slow to correct when GPS says it is wrong.
- **Correctly tuned τ** : the estimate (black) stays close to the true signal (red) — the GPS prevents long-term drift while the velocity sensor provides the fast, smooth dynamics between GPS fixes. The error is much smaller than the raw measurement noise at all times.

How to choose τ in practice

There is no single formula for τ — it depends on the relative noise levels of your two sensors. Two practical guidelines:

- **Rule of thumb:** set the crossover frequency $\omega_c = 1/\tau$ to be in the frequency band where your GPS noise and your velocity-integration drift are roughly equal in magnitude. Below this frequency, GPS is more reliable; above it, velocity integration is cleaner.
- **Trial and adjust:** start with a τ of a few seconds (for a GPS + IMU system), simulate or test on real data, and observe whether the estimate lags GPS errors (too large τ) or is too noisy (too small τ). Adjust τ until the black curve in the performance plot approximates the red curve well.

If you have careful noise models for both sensors (variances and spectral densities), use the Kalman filter instead — it computes the optimal crossover automatically from those models and adapts it over time. The complementary filter with fixed τ is optimal only if the noise levels stay constant, which in practice they often do not.

Section 3 — Key Takeaways

- A **complementary filter** fuses two sensors by splitting the frequency spectrum between them: $z = F_1(s)y_1 + F_2(s)y_2$ with $F_1(s) = I - F_2(s)$. One sensor covers low frequencies; the other covers high frequencies; together they cover everything exactly once.
- Choose $F_2(s)$ as a low-pass filter (e.g. $\frac{1}{\tau s + 1}$) and $F_1(s)$ follows automatically as its high-pass complement $\frac{\tau s}{\tau s + 1}$.
- For the velocity+position example: $\hat{y} = \frac{1}{\tau s + 1}(\tau v + y)$. GPS prevents drift at low frequencies; velocity integration gives smooth, fast updates at high frequencies. The time constant τ sets the crossover between them.
- The discrete-time implementation uses the **Tustin (bilinear) approximation** ($s \leftarrow \frac{2}{T_s} \frac{z-1}{z+1}$), giving the recursion:

$$\hat{y}_k = \frac{1}{\alpha_1}(-\alpha_0 \hat{y}_{k-1} + \beta \frac{T_s}{2} v_k + \beta \frac{T_s}{2} v_{k-1} + y_k + y_{k-1})$$

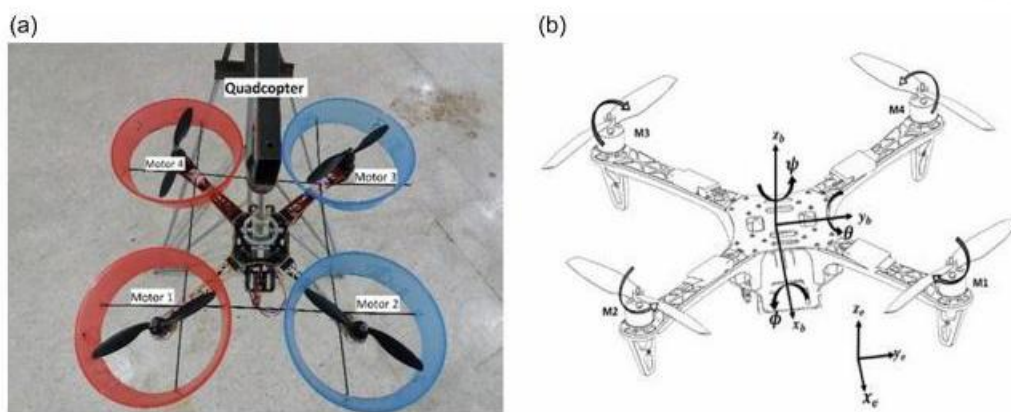
with $\alpha_1 = \frac{\tau T_s}{2} + 1$, $\alpha_0 = 1 - \frac{\tau T_s}{2}$, $\beta = \frac{\tau T_s}{2}$.

- The complementary filter is a computationally trivial alternative to the Kalman filter when: (1) two sensors split cleanly by frequency, and (2) noise levels are roughly constant so a fixed τ is acceptable. The KF is preferable when optimal weighting or time-varying noise is important.

Section 4 Attitude Determination & Orientation Representation

So far the lecture has dealt with sensor fusion in the context of quantities that live in ordinary Euclidean space — position, velocity, altitude. We now turn to a different and more geometrically subtle problem: **attitude determination** — estimating the 3D orientation of a rigid body (a drone, a phone, a robot arm) from sensor measurements. This is harder than it first appears, both because the sensors are imperfect in complementary ways, and because orientation itself does not live in a simple flat space.

4.1 The Problem — Why IMU Sensors Alone Are Insufficient



A common situation where sensor fusion gives clear benefits is *attitude determination*, for instance using IMUs.

- ▶ Gyroscopes measure rate of rotation directly, typically in rad/s.
- ▶ Accelerometers measure the sum of gravity and acceleration in m/s^2 .
- ▶ Magnetometers measure the magnetic field, typically in μT .

Figure 13: Attitude determination on a quadcopter. (a) Physical quadcopter with four motors. (b) Schematic showing the body-fixed axes (x_b, y_b, z_b) and the sensors mounted to the frame. The goal is to estimate the **attitude** of the body — the orientation of the body-fixed frame relative to a fixed world frame — from the IMU sensor readings. The three sensor types are gyroscopes (angular rate), accelerometers (specific force), and magnetometers (magnetic field).

An **Inertial Measurement Unit (IMU)** typically contains three types of sensor, each measuring a different physical quantity:

- **Gyroscope** — measures the *angular rate* of the body, typically in rad/s. Integrating the angular rate gives orientation — but any small bias in the gyroscope reading accumulates over time into a large orientation error (**drift**). Good at tracking fast rotations; fails over long time scales without correction.

- **Accelerometer** — measures the *specific force*: the vector sum of all non-gravitational forces per unit mass acting on the sensor, in m/s^2 . In steady state (hovering, standing still), the accelerometer reads only the gravity vector, which points straight down in the world frame. The direction of this gravity reading tells you roll and pitch. But during any acceleration, the external acceleration adds to gravity and corrupts the attitude estimate.
- **Magnetometer** — measures the ambient magnetic field vector, typically in μT . In steady state this vector points approximately toward magnetic north, giving a reference for the yaw angle. But the magnetometer is sensitive to nearby ferromagnetic objects and electric currents (motors, wiring), which can deflect the measured field far from the true Earth field.

Why each sensor alone is insufficient — the complementary structure

Each sensor has a strength the others lack, and a weakness the others compensate for:

Sensor	Strength	Weakness
Gyroscope	Fast, smooth — tracks rapid rotations with high bandwidth	Drifts over time — any bias integrates into unbounded error
Accelerometer	Stable long-term gravity reference — no drift for roll and pitch	Corrupted during manoeuvres — cannot separate gravity from linear acceleration
Magnetometer	Stable long-term yaw reference (magnetic north) — no drift	Disturbed by nearby metal and electronics — unreliable near motors

The complementary structure is clear: the gyroscope is reliable at *high frequencies* (fast rotations) but fails at low frequencies (drift); the accelerometer and magnetometer provide stable *low-frequency* references but are corrupted at high frequencies (vibration, acceleration). This is exactly the complementary-filter split from Section 3, applied now to orientation rather than position. The Madgwick and Mahoney filters in Sections 6 and 7 exploit this structure explicitly.

4.2 Euler Angles — Describing Orientation with Three Angles

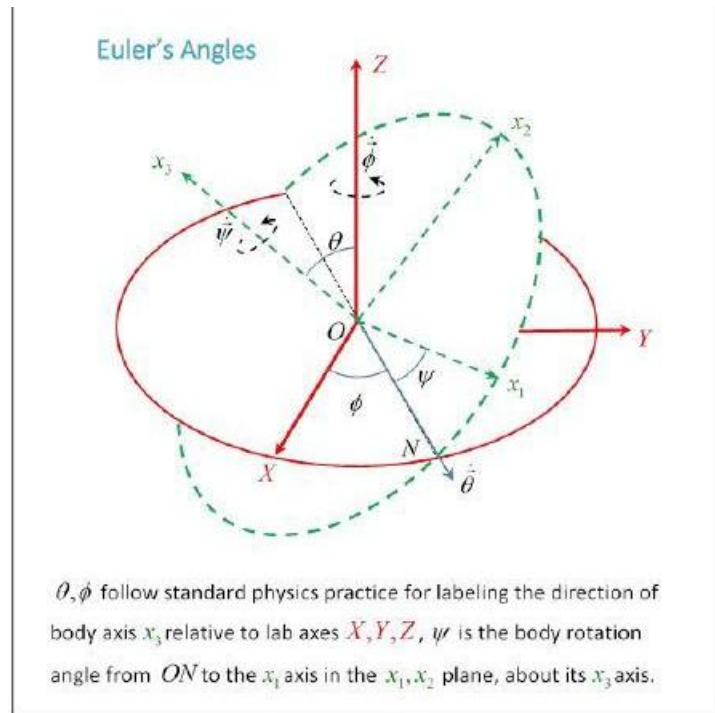


Figure 14: Euler angles. The orientation of the body frame (x_1, x_2, x_3) relative to the lab frame (X, Y, Z) is described by three successive rotations. ϕ (roll): rotation about the body x_1 axis. θ (pitch): rotation about the intermediate x_2 axis. ψ (yaw): the rotation angle from the reference line ON to the x_1 axis in the x_1 - x_2 plane, measured about the x_3 axis. The dashed lines show the intermediate frames created by each successive rotation.

The most intuitive way to describe the orientation of a rigid body in 3D is with **Euler angles**: three angles that parameterise three successive rotations bringing the world frame into alignment with the body frame.

For aerospace and robotics the standard convention is:

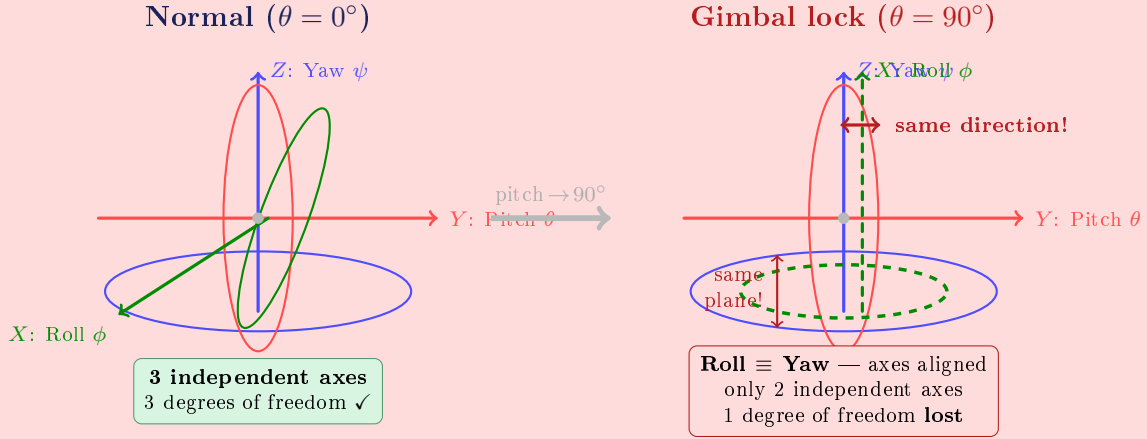
- **Roll** ϕ : rotation about the forward (x) axis — tilting left or right. Range: $[-180^\circ, 180^\circ]$.
- **Pitch** θ : rotation about the right (y) axis — nose up or nose down. Range: $[-90^\circ, 90^\circ]$.
- **Yaw** ψ : rotation about the vertical (z) axis — turning left or right like a compass heading. Range: $[-180^\circ, 180^\circ]$.

The order of rotations matters — Euler angles are not commutative

Unlike adding numbers, rotations in 3D do not commute. Rotating 90° about x first then 90° about z gives a different result than the same rotations in reverse order. Euler angles must always be applied in a fixed, agreed-upon sequence. The most common convention in aerospace is **ZYX** (yaw first, then pitch, then roll), also called the *Tait-Bryan* convention. Always check which convention a paper or datasheet uses — mixing them gives wrong results with no obvious error message.

Gimbal lock — the fundamental problem with Euler angles

Euler angles have a critical failure mode called **gimbal lock**. It occurs when the pitch angle reaches $\pm 90^\circ$ (nose straight up or straight down). At this configuration, the roll axis and the yaw axis become aligned — two of the three rotations now do the same thing and one degree of freedom is lost.



In practice this means:

- The Euler angle representation becomes **singular** — infinitely many (ϕ, θ, ψ) combinations describe the same physical orientation.
- **Numerical derivatives blow up** — the kinematic equations relating angular velocity ω to $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ contain $1/\cos \theta$ terms which diverge as $\theta \rightarrow \pm 90^\circ$.
- Any filter propagating Euler angles will fail or produce nonsense near this singularity.

For a drone doing aerobatics, a robot arm that can reach above its head, or any system that passes through $\theta = \pm 90^\circ$, Euler angles are unsuitable. **Quaternions** (Section 4.3) were developed specifically to avoid this singularity.

4.3 Quaternions — A Singularity-Free Representation

A quaternion is a 4-element vector that encodes any 3D rotation as one **rotation axis** $\hat{n}$ and one **rotation angle** α :

$$\mathbf{q} = \begin{bmatrix} q_0 \\ \vec{q} \end{bmatrix} = \begin{bmatrix} q_0 \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} \cos(\alpha/2) \\ \sin(\alpha/2) n_x \\ \sin(\alpha/2) n_y \\ \sin(\alpha/2) n_z \end{bmatrix} \in \mathbb{R}^4$$

The scalar part q_0 encodes how large the rotation is; the vector part $\vec{q}$ encodes which axis it is about. For it to represent a pure rotation (no scaling), we **define and enforce** the unit-length constraint:

$$\|\mathbf{q}\| = \sqrt{q_0^2 + q_x^2 + q_y^2 + q_z^2} = 1$$

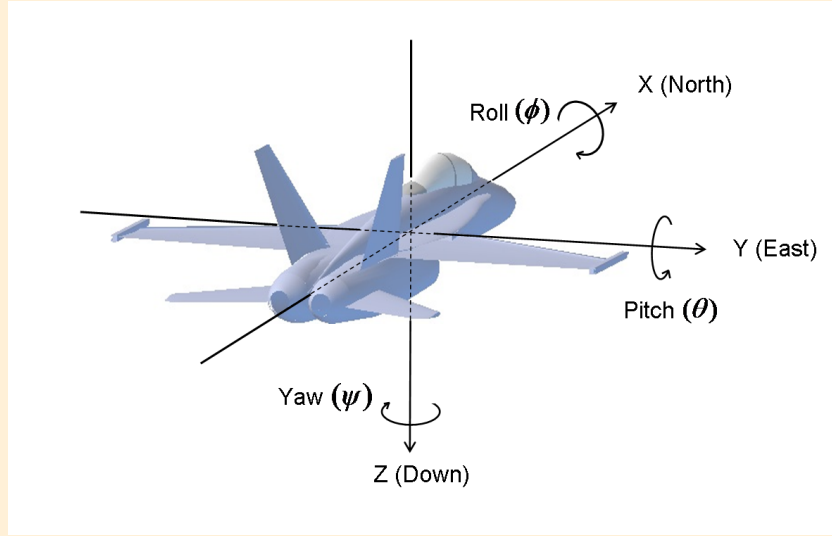
This is not automatically true — it is a requirement we impose by construction. When we build $\mathbf{q}$ from an axis $\hat{n}$ and angle α using the formula above, the result satisfies $\|\mathbf{q}\| = 1$ exactly (because $\cos^2(\alpha/2) + \sin^2(\alpha/2) = 1$). But when a filter integrates the quaternion

numerically over time, floating- point rounding slowly pushes the norm away from 1. The fix is a periodic one-line renormalisation:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}$$

Without this step the quaternion drifts off the unit sphere and stops representing a valid rotation.

Concrete example — what do the four numbers actually mean?



Suppose a drone needs to rotate 90° about the world z -axis (a pure yaw turn). The rotation axis is $\hat{n} = (0, 0, 1)$ and the angle is $\alpha = 90^\circ$. Plugging in:

$$\mathbf{q} = \begin{bmatrix} \cos(45^\circ) \\ \sin(45^\circ) \cdot 0 \\ \sin(45^\circ) \cdot 0 \\ \sin(45^\circ) \cdot 1 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ 0 \\ 0 \\ \frac{\sqrt{2}}{2} \end{bmatrix} \approx \begin{bmatrix} 0.707 \\ 0 \\ 0 \\ 0.707 \end{bmatrix}$$

Reading the four numbers:

- $q_x = 0, q_y = 0$: no x or y component in the axis — pure z -axis rotation.
- $q_z = 0.707 = \sin(45^\circ)$: the rotation axis points entirely along z (yaw axis).
- $q_0 = 0.707 = \cos(45^\circ)$: see below — this is the confusing part.

Verify unit norm: $0.707^2 + 0^2 + 0^2 + 0.707^2 = 0.5 + 0.5 = 1 \checkmark$

Why does $q_0 = 0.707$ mean 90° , and $q_0 = 0$ mean 180° ? This trips everyone up because q_0 works *backwards* from what feels natural. $q_0 = \cos(\alpha/2)$, and cosine *decreases* as the angle grows: the bigger the rotation, the *smaller* q_0 gets.

Think of it with a barrel roll. You start level (0° roll, $q_0 = 1$). As you roll over, q_0 falls. When you are completely upside-down (180° roll), $q_0 = 0$. That is the most “different” orientation from level — not zero rotation, but the *maximum*. Roll all the way back to level (360°) and q_0 reaches -1 , then the sign flips back to $+1$ at the next full turn.

Situation	α	$q_0 = \cos(\alpha/2)$
Not rotated at all (identity)	0°	1
Quarter turn	90°	0.707
Half turn — fully flipped	180°	0
Full turn — back to start	360°	-1

So q_0 is a measure of **how close to the identity** you are: $q_0 = 1$ means sitting still, $q_0 = 0$ means halfway around, $q_0 = -1$ means a full turn with a sign flip.

The three reference quaternions (all yaw rotations, axis = z):

$$\mathbf{q}_{90^\circ} = \begin{bmatrix} 0.707 \\ 0 \\ 0 \\ 0.707 \end{bmatrix}, \quad \mathbf{q}_{180^\circ} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{q}_{\text{identity}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Why $\alpha/2$ and not α ? In the rotation formula $\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$ (Section 4.5), $\mathbf{q}$ acts twice — left and right. Each application contributes $\alpha/2$; together they give α . Storing α directly would produce a double rotation.

q_0 works backwards — bigger rotation means smaller q_0

$q_0 = \cos(\alpha/2)$ *decreases* as the rotation grows. The bigger the rotation, the smaller q_0 gets — not the larger.

Think of a barrel roll: you start level ($q_0 = 1$). As you roll over, q_0 falls. Completely upside-down (180°): $q_0 = 0$ — the most “different” orientation from level. Back to level after a full 360° : $q_0 = -1$, sign-flipped.

Does the filter reset q_0 back to 1 after a full turn? No — never. $(-1, 0, 0, 0)$ and $(1, 0, 0, 0)$ are physically identical (same orientation, just opposite sign). The filter keeps running continuously; no reset happens.

Sitting still = quaternion not changing over time, regardless of what the four numbers are. A quaternion frozen at $(-1, 0, 0, 0)$ means sitting still at the reference orientation. A quaternion frozen at $(0.707, 0, 0, 0.707)$ means sitting still at 90° yaw from reference.

$(1, 0, 0, 0)$ is not a fixed physical direction — it is relative to your initial state

The identity quaternion does not mean “level”, “pointing north”, or any fixed world direction. It simply means **no rotation from wherever the filter started**.

- Start level, pointing north $\rightarrow (1, 0, 0, 0)$ means level and pointing north.
- Start pointing straight up $\rightarrow (1, 0, 0, 0)$ means pointing straight up.
- Start banked 45° right $\rightarrow (1, 0, 0, 0)$ means banked 45° right.

All three read $(1, 0, 0, 0)$ but describe completely different physical orientations. This

is why IMU filters must be **initialised carefully** — whatever orientation the body is in when the filter starts becomes the reference that all subsequent quaternions are measured against.

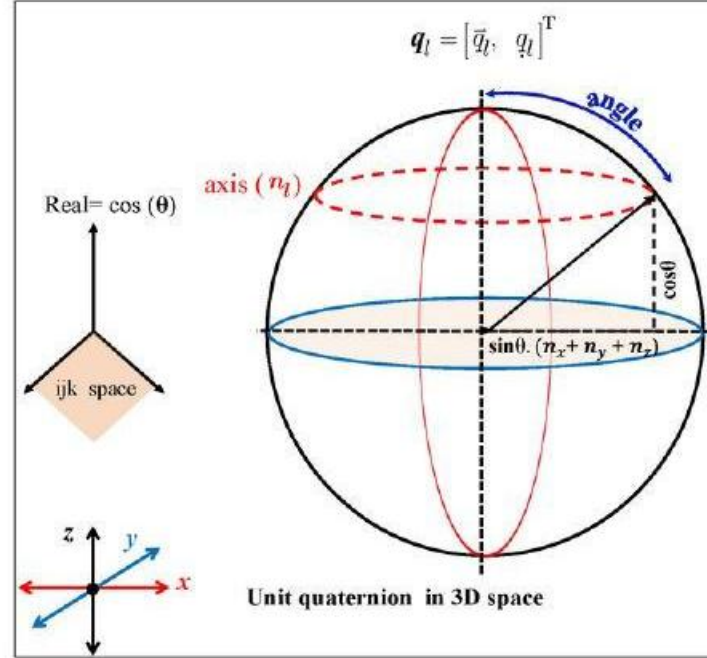


Figure 15: Unit quaternion sphere. Every point on the surface of this sphere is one specific 3D rotation. **Vertical axis** ($\text{Real} = q_0 = \cos(\alpha/2)$): encodes the rotation size — top ($q_0 = 1$) is the identity (no rotation), equator ($q_0 = 0$) is a 180° half-turn, bottom ($q_0 = -1$) is a full 360° turn (same physical orientation as the identity, but negated quaternion). **Horizontal disc** (ijk space): the 3D space of the imaginary part $\vec{q}$ — the direction in this disc is the rotation axis η_l , and the distance from the centre is $\sin(\alpha/2)$. **Opposite points** $\mathbf{q}$ and $-\mathbf{q}$ always represent the same physical rotation — every orientation has two quaternion representations.

Why quaternions avoid gimbal lock

Euler angles use 3 numbers for 3D orientation. The problem: the space of all 3D rotations ($SO(3)$) cannot be covered by 3 smooth coordinates without at least one singularity — gimbal lock is that unavoidable breakdown.

Unit quaternions use 4 numbers with 1 constraint ($\|\mathbf{q}\| = 1$), giving 3 effective free parameters — same count as Euler angles, but the constraint surface is a smooth sphere with no singular points anywhere. Integration through any orientation works cleanly. The only housekeeping needed: renormalise $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$ periodically, since numerical drift slowly pushes the quaternion off the unit sphere.

4.4 Converting Quaternions to Euler Angles

Assuming a quaternion is properly normalized, the corresponding Euler angles can be obtained via the relations:

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \text{atan2}(2(q_0 q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)) \\ -\pi/2 + 2 \text{atan2}\left(\sqrt{1 + 2(q_0 q_y - q_x q_z)}, \sqrt{1 - 2(q_0 q_y - q_x q_z)}\right) \\ \text{atan2}(2(q_0 q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \end{bmatrix}$$

Figure 16: Quaternion-to-Euler conversion. Given a unit quaternion (q_0, q_x, q_y, q_z) , the corresponding ZYX Tait-Bryan Euler angles (roll ϕ , pitch θ , yaw ψ) are recovered by three `atan2` expressions. The filters in Sections 6 and 7 work in quaternion space internally; Euler angles are only computed here at the output for display.

The attitude filters work internally with quaternions. For human-readable output the quaternion is converted to Euler angles once at the end:

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \text{atan2}(2(q_0 q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)) \\ -\frac{\pi}{2} + 2 \text{atan2}\left(\sqrt{1 + 2(q_0 q_y - q_x q_z)}, \sqrt{1 - 2(q_0 q_y - q_x q_z)}\right) \\ \text{atan2}(2(q_0 q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \end{bmatrix}$$

Symbol	Name	Meaning
ϕ	Roll	Rotation about the body x -axis (forward axis). Positive = right side down.
θ	Pitch	Rotation about the body y -axis (right axis). Positive = nose up.
ψ	Yaw	Rotation about the body z -axis (down axis). Positive = clockwise from above.
<code>atan2(y, x)</code>	Four-quadrant arctangent	Returns the angle in $(-\pi, \pi]$ whose tangent is y/x , using the signs of both arguments to choose the correct quadrant. Handles $x = 0$ cleanly and covers the full 360° range, unlike <code>arctan(y/x)</code> .

Which quaternion components control which angle

Each Euler angle is extracted from a different subset of quaternion components, determined by which body axis it rotates about:

- **Roll** ϕ : terms q_0q_x and q_yq_z both involve q_x — the imaginary component along the rotation axis for roll (x -axis).
- **Pitch** θ : term $q_0q_y - q_xq_z$ involves q_y — the y -axis component. The double-`atan2` with square roots replaces a simple `arcsin` to avoid the $\pm 90^\circ$ discontinuity that would otherwise appear at the boundary of the `arcsin` range.
- **Yaw** ψ : terms q_0q_z and q_xq_y involve q_z — the z -axis component.

This is useful for debugging: if yaw suddenly goes wrong while roll and pitch are fine, the error is in the q_z component of your quaternion. In practice use a library function (`quat2eul` in MATLAB) — knowing the structure tells you which component to inspect, not how to reimplement the formula.

4.5 Rotating a Vector with a Quaternion

the formula

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_0r_0 - \vec{q} \cdot \vec{r} \\ q_0\vec{r} - r_0\vec{q} + \vec{q} \times \vec{r} \end{bmatrix}$$

where $\cdot$ and $\times$ indicate the usual three-dimensional dot and vector cross products.

- The canonical way to rotate a three-dimensional vector $\vec{v}$ by a quaternion $\mathbf{q}$ defining an Euler rotation is via the formula

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$$

where $\mathbf{v} = (0, \vec{v})$ is a quaternion containing the embedded vector $\vec{v}$, $\mathbf{q}^* = (q_0, -\vec{q})$ is the *conjugate* of $\mathbf{q}$, and $\mathbf{v}' = (0, \vec{v}')$ gives the rotated vector $\vec{v}'$.

- An alternative approach is to apply the equations

$$\begin{aligned} \vec{t} &= 2\vec{q} \times \vec{v} \\ \vec{v}' &= \vec{v} + q_0\vec{t} + \vec{q} \times \vec{t}. \end{aligned}$$

- Note that

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|}.$$

Figure 17: Quaternion multiplication and rotation. **Top:** the general quaternion product $\mathbf{q} \otimes \mathbf{r}$. **Middle:** rotating vector $\vec{v}$ by $\mathbf{q}$ via $\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$. **Bottom:** the efficient two-cross-product alternative that avoids computing two full quaternion products. **Far right:** the quaternion inverse $\mathbf{q}^{-1} = \mathbf{q}^*/\|\mathbf{q}\|$.

Quaternion multiplication.

Given two quaternions written out in full:

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_x \\ q_y \\ q_z \end{bmatrix}, \quad \mathbf{r} = \begin{bmatrix} r_0 \\ r_x \\ r_y \\ r_z \end{bmatrix}$$

their product is:

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_0 r_0 - \vec{q} \cdot \vec{r} \\ q_0 \vec{r} + r_0 \vec{q} + \vec{q} \times \vec{r} \end{bmatrix} = \begin{bmatrix} q_0 r_0 - q_x r_x - q_y r_y - q_z r_z \\ q_0 r_x + q_x r_0 + q_y r_z - q_z r_y \\ q_0 r_y - q_x r_z + q_y r_0 + q_z r_x \\ q_0 r_z + q_x r_y - q_y r_x + q_z r_0 \end{bmatrix}$$

Symbol	Name	Meaning
$\otimes$	Quaternion product	Non-commutative: $\mathbf{q} \otimes \mathbf{r} \neq \mathbf{r} \otimes \mathbf{q}$ in general, just like matrix multiplication.
$\vec{q} \cdot \vec{r}$	Dot product	$q_x r_x + q_y r_y + q_z r_z$ — scalar.
$\vec{q} \times \vec{r}$	Cross product	$(q_y r_z - q_z r_y, q_z r_x - q_x r_z, q_x r_y - q_y r_x)^\top$ — vector, anti-symmetric: $\vec{q} \times \vec{r} = -\vec{r} \times \vec{q}$. This anti-symmetry is why $\otimes$ is non-commutative.

Rotating a vector with a quaternion.

To rotate 3D vector $\vec{v}$ by the rotation encoded in unit quaternion $\mathbf{q}$, first form three quaternions side by side:

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_x \\ q_y \\ q_z \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 0 \\ v_x \\ v_y \\ v_z \end{bmatrix}, \quad \mathbf{q}^* = \begin{bmatrix} q_0 \\ -q_x \\ -q_y \\ -q_z \end{bmatrix}$$

$\mathbf{v}$ is the vector embedded as a **pure quaternion** (zero scalar part). $\mathbf{q}^*$ is the **conjugate** of $\mathbf{q}$: same scalar, negated vector part. Then apply the double product:

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^* \implies \mathbf{v}' = \begin{bmatrix} 0 \\ v'_x \\ v'_y \\ v'_z \end{bmatrix}$$

The result $\mathbf{v}'$ is also a pure quaternion; its vector part (v'_x, v'_y, v'_z) is the rotated vector.

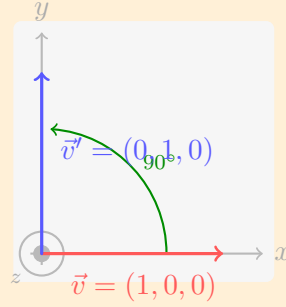
Why the double product gives the correct rotation

For a unit quaternion, the conjugate equals the inverse: $\mathbf{q}^* = \mathbf{q}^{-1}$, so $\mathbf{q} \otimes \mathbf{q}^* = (1, \vec{0})$ (identity). The double-sided product $\mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$ is an algebraic *conjugation* with three guaranteed properties:

- If $\mathbf{v}$ is pure (zero scalar), so is $\mathbf{v}'$ — the scalar cancels. The output is always a valid 3D vector.
- The magnitude is preserved: $|\vec{v}'| = |\vec{v}|$ — pure rotation, no scaling.

- The rotation angle is exactly α : each factor contributes $\alpha/2$, together giving α .

Example — rotate $\vec{v} = (1, 0, 0)^\top$ by 90° about $\hat{z}$:



The three quaternions involved:

$$\mathbf{q} = \begin{bmatrix} \cos 45^\circ \\ 0 \\ 0 \\ \sin 45^\circ \end{bmatrix} = \begin{bmatrix} 0.707 \\ 0 \\ 0 \\ 0.707 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{q}^* = \begin{bmatrix} 0.707 \\ 0 \\ 0 \\ -0.707 \end{bmatrix}$$

Computing $\mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$ gives:

$$\mathbf{v}' = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \implies \vec{v}' = (0, 1, 0)^\top$$

The red vector along x rotated 90° about z and now points along y (blue). Correct.

Efficient alternative rotation formula.

Two full quaternion products is expensive on embedded hardware. Given:

$$\mathbf{q} = \begin{bmatrix} q_0 \\ \vec{q} \end{bmatrix}, \quad \vec{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}$$

An equivalent formula using only vector operations:

$$\vec{\ell} = 2 \vec{q} \times \vec{v}, \quad \vec{v}' = \vec{v} + q_0 \vec{\ell} + \vec{q} \times \vec{\ell}$$

Two cross products and a few additions — preferred in real-time embedded implementations.

Verification with the same example ($\vec{v} = (1, 0, 0)^\top$, 90° about z , so $q_0 = 0.707$, $\vec{q} = (0, 0, 0.707)$):

Step 1 — compute $\vec{\ell}$:

$$\vec{q} \times \vec{v} = \begin{bmatrix} 0 \\ 0 \\ 0.707 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot 0 - 0.707 \cdot 0 \\ 0.707 \cdot 1 - 0 \cdot 0 \\ 0 \cdot 0 - 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.707 \\ 0 \end{bmatrix} \implies \vec{\ell} = 2 \begin{bmatrix} 0 \\ 0.707 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.414 \\ 0 \end{bmatrix}$$

Step 2 — compute $\vec{v}'$:

$$q_0 \vec{\ell} = 0.707 \begin{bmatrix} 0 \\ 1.414 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\vec{q} \times \vec{\ell} = \begin{bmatrix} 0 \\ 0 \\ 0.707 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1.414 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot 0 - 0.707 \cdot 1.414 \\ 0.707 \cdot 0 - 0 \cdot 0 \\ 0 \cdot 1.414 - 0 \cdot 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}$$

$$\vec{v}' = \vec{v} + q_0 \vec{\ell} + \vec{q} \times \vec{\ell} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \checkmark$$

Same result as the double-product method — $(0, 1, 0)^\top$ as expected.

Quaternion inverse.

Given:

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_x \\ q_y \\ q_z \end{bmatrix}$$

For any quaternion:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|} = \frac{1}{\|\mathbf{q}\|} \begin{bmatrix} q_0 \\ -q_x \\ -q_y \\ -q_z \end{bmatrix}$$

For a unit quaternion ($\|\mathbf{q}\| = 1$, which always is given our constraint that it needs to be from earlier), this simplifies to:

$$\mathbf{q}^{-1} = \mathbf{q}^* = \begin{bmatrix} q_0 \\ -q_x \\ -q_y \\ -q_z \end{bmatrix}$$

The inverse represents the *opposite* rotation: if $\mathbf{q}$ rotates the world frame into the body frame, $\mathbf{q}^{-1}$ rotates back.

Common misconception — $180^\circ + 180^\circ$ does not mean a double rotation

It is tempting to think that rotating 180° about y and then 180° about z results in a total of $\|\mathbf{q}\| = 2$ **This is wrong**. Rotations in 3D do not add like numbers — they compose, and the result depends on the axes involved.

The three quaternions involved:

$$\mathbf{q}_y = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad (180^\circ \text{ about } y) \quad \mathbf{q}_z = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (180^\circ \text{ about } z)$$

Computing the composition $\mathbf{q}_y \otimes \mathbf{q}_z$ (apply z first, then y):

$$\mathbf{q}_y \otimes \mathbf{q}_z = \begin{bmatrix} 0\cdot0 - 0\cdot0 - 1\cdot0 - 0\cdot1 \\ 0\cdot0 + 0\cdot0 + 1\cdot1 - 0\cdot0 \\ 0\cdot0 - 0\cdot1 + 1\cdot0 + 0\cdot0 \\ 0\cdot1 + 0\cdot0 - 1\cdot0 + 0\cdot0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad (180^\circ \text{ about } x)$$

All three side by side:

$$\underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}}_{180^\circ \text{ about } z} \quad \underbrace{\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}}_{180^\circ \text{ about } y} \quad \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}}_{180^\circ \text{ about } y \text{ then } z}$$

All three have $q_0 = 0$ — all three are 180° rotations. The vector part (q_x, q_y, q_z) tells you *which* axis: the two separate 180° rotations composed into a *third* 180° rotation, this time about the x -axis. The quaternion only stores the final equivalent result — it has no memory of how many steps were taken to get there.

Section 4 — Key Takeaways

- IMU attitude estimation needs **three sensor types**: gyroscope (fast, drifts), accelerometer (gravity reference, disturbed by manoeuvres), magnetometer (yaw reference, disturbed by magnetic fields). Each covers the other's weakness — the same complementary structure as Section 3, applied to orientation.
- **Euler angles** (ϕ, θ, ψ) : intuitive but suffer from **gimbal lock** at $\theta = \pm 90^\circ$ where two rotation axes align, the representation is singular and numerical derivatives diverge.
- **Unit quaternions** $\mathbf{q} = (q_0, \vec{q}) \in \mathbb{R}^4$, $\|\mathbf{q}\| = 1$: rotation of angle α about unit axis $\hat{n}$ is $\mathbf{q} = (\cos(\alpha/2), \sin(\alpha/2) \hat{n})$. No singularity at any orientation.
- **Quaternion-to-Euler**: three `atan2` expressions. Filters run in quaternion space; Euler angles computed at output only.
- **Quaternion product**: $\mathbf{q} \otimes \mathbf{r} = (q_0 r_0 - \vec{q} \cdot \vec{r}, q_0 \vec{r} + r_0 \vec{q} + \vec{q} \times \vec{r})$. Non-commutative — order matters.
- **Rotating** $\vec{v}$ by $\mathbf{q}$: $\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*$, or efficiently: $\vec{\ell} = 2\vec{q} \times \vec{v}$, $\vec{v}' = \vec{v} + q_0 \vec{\ell} + \vec{q} \times \vec{\ell}$.
- For unit quaternions: $\mathbf{q}^{-1} = \mathbf{q}^* = (q_0, -\vec{q})$.

Section 5 IMU Measurement Models & Quaternion Kinematics

We now have the mathematical tools for representing orientation as a quaternion. This section builds the *models* that connect the IMU sensor readings to that quaternion — the gyroscope model that drives the quaternion forward in time, and the accelerometer/magnetometer models that provide correction references. These models are the inputs to the Madgwick and Mahoney filters in Sections 6 and 7.

5.1 IMU Measurement Models

- Gyroscope measurement model²:

$$\begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} \bar{\omega}_x \\ \bar{\omega}_y \\ \bar{\omega}_z \end{bmatrix} + \begin{bmatrix} \omega_{x,b} \\ \omega_{y,b} \\ \omega_{z,b} \end{bmatrix} + \begin{bmatrix} \nu_{\omega,x} \\ \nu_{\omega,y} \\ \nu_{\omega,z} \end{bmatrix}$$

where $(\cdot)_b$ denotes bias and $\bar{\cdot}$ denotes ‘true’ angular rotation speed;

- Accelerometer measurement model:

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = \begin{bmatrix} \bar{a}_x \\ \bar{a}_y \\ \bar{a}_z \end{bmatrix} + \begin{bmatrix} a_{x,b} \\ a_{y,b} \\ a_{z,b} \end{bmatrix} + \begin{bmatrix} \nu_{a,x} \\ \nu_{a,y} \\ \nu_{a,z} \end{bmatrix}$$

where $\bar{a}$ is the sum of “gravity” and external acceleration;

- Magnetometer measurement model:

$$\begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix} = \begin{bmatrix} \bar{s}_x \\ \bar{s}_y \\ \bar{s}_z \end{bmatrix} + \begin{bmatrix} s_{x,b} \\ s_{y,b} \\ s_{z,b} \end{bmatrix} + \begin{bmatrix} \nu_{s,x} \\ \nu_{s,y} \\ \nu_{s,z} \end{bmatrix}$$

²Note that the notation is changed from the one used in Ludwig and Burnham's paper

Figure 18: Measurement models for all three IMU sensor types. Each sensor reading is the sum of three terms: the **true** physical quantity (tilde/bar notation), a **bias** (subscript b), and additive **noise** (ν). The gyroscope measures angular rate; the accelerometer measures the sum of gravity and external acceleration; the magnetometer measures the ambient magnetic field.

All three IMU sensors share the same model structure:

$$\text{measured} = \text{true value} + \text{bias} + \text{noise}$$

Gyroscope (angular rate, rad/s):

$$\begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \underbrace{\begin{bmatrix} \tilde{\omega}_x \\ \tilde{\omega}_y \\ \tilde{\omega}_z \end{bmatrix}}_{\text{true rate}} + \underbrace{\begin{bmatrix} \omega_{x,b} \\ \omega_{y,b} \\ \omega_{z,b} \end{bmatrix}}_{\text{bias}} + \underbrace{\begin{bmatrix} \nu_{\omega,x} \\ \nu_{\omega,y} \\ \nu_{\omega,z} \end{bmatrix}}_{\text{noise}}$$

Accelerometer (specific force, m/s²):

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = \underbrace{\begin{bmatrix} \bar{a}_x \\ \bar{a}_y \\ \bar{a}_z \end{bmatrix}}_{\text{true specific force}} + \underbrace{\begin{bmatrix} a_{x,b} \\ a_{y,b} \\ a_{z,b} \end{bmatrix}}_{\text{bias}} + \underbrace{\begin{bmatrix} \nu_{a,x} \\ \nu_{a,y} \\ \nu_{a,z} \end{bmatrix}}_{\text{noise}}$$

Magnetometer (magnetic field, μT):

$$\begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix} = \underbrace{\begin{bmatrix} \bar{s}_x \\ \bar{s}_y \\ \bar{s}_z \end{bmatrix}}_{\text{true field}} + \underbrace{\begin{bmatrix} s_{x,b} \\ s_{y,b} \\ s_{z,b} \end{bmatrix}}_{\text{bias}} + \underbrace{\begin{bmatrix} \nu_{s,x} \\ \nu_{s,y} \\ \nu_{s,z} \end{bmatrix}}_{\text{noise}}$$

Symbol	Name	Meaning
$\tilde{\omega}$	True angular rate	The real rotation rate of the body. Tilde $\tilde{\cdot}$ denotes the true unknown value.
ω_b	Gyroscope bias	A slowly drifting offset — the gyroscope outputs a small nonzero value even when perfectly still. The main cause of orientation drift over time.
ν_ω	Gyroscope noise	Fast zero-mean random noise. Modelled as Gaussian white noise.
$\bar{a}$	True specific force	The true vector sum of gravity and any external linear acceleration. Bar $\bar{\cdot}$ denotes the true (noise- and bias-free) value. In steady state: $\bar{a} = (0, 0, -g)^\top$ (NED).
a_b	Accelerometer bias	Constant or slowly varying measurement offset.
$\bar{s}$	True magnetic field	The true ambient magnetic field vector in the body frame.
s_b	Magnetometer bias	Hard-iron and soft-iron distortion — offsets caused by permanent magnets and ferromagnetic material near the sensor.

Bias vs noise — why bias is the bigger problem

Both corrupt the reading, but cause very different errors:

- **Noise** is zero-mean and fast-varying. When integrated over time, positive and negative errors partially cancel. The orientation error grows as $\sqrt{t}$ — a random walk.
- **Bias** is a constant offset always in the same direction. When integrated, it accumulates monotonically: every second adds the same wrong increment. The orientation error grows as t — linearly. A gyroscope bias of just $0.1^\circ/\text{s}$ produces 6° of heading error after one minute and a full 360° after one hour.

This is why the Madgwick and Mahoney filters explicitly estimate and subtract the gyroscope bias ω_b as part of their update. Correcting bias is the single most important step for long-term stability.

What the accelerometer actually measures — and why gravity is useful

The accelerometer reads the **specific force**: all non-gravitational forces per unit mass. A body sitting still on a table feels the table pushing up against gravity — the accelerometer reads that upward contact force, not gravity directly.

In steady state (hovering, stationary, slow movement):

$$\bar{a} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \quad (\text{NED frame, } g \approx 9.81 \text{ m/s}^2)$$

Only gravity is present. The *direction* of this gravity vector in the body frame directly reveals roll and pitch: tilt the body and the measured gravity direction tilts with it.

During manoeuvres, linear acceleration adds to gravity and the reading is no longer a pure gravity reference.

This is the key subtlety: we are not using the accelerometer to measure how much the drone is accelerating — that is not the goal here at all. We are using it as a **gravity direction sensor**. In steady state, the only force it feels is gravity, so the direction of its reading tells us which way is “down” in the body frame, and from that we extract roll and pitch.

But when the drone actually accelerates (thrusts sideways, brakes, tilts aggressively), a second force — the linear acceleration — adds on top of gravity. The combined vector no longer points straight down; it points in some mixed direction that is *not* the true gravity direction. If the filter uses that corrupted direction as a gravity reference, it will think the body is tilted when it is not, and introduce a wrong attitude correction.

So: trusted less during acceleration not because we don’t want to measure acceleration — but because the gravity direction we need for attitude correction is hidden inside a mixture of gravity and linear acceleration, and we can no longer separate them. A simple check: if $\|\bar{a}\| \neq g$, the body is accelerating and the accelerometer reading is a corrupted gravity reference.

5.2 Quaternion Kinematic Equation

- Kinematic equation:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}$$

where $\mathbf{q}$ represents orientation and $\boldsymbol{\omega} = (0, \vec{\omega})$ represents angular velocity.

- Acceleration quaternion representation rotated by $\mathbf{q}$:

$$\mathbf{a} = \mathbf{q}^{-1} \otimes \begin{bmatrix} 0 \\ \vec{a} \end{bmatrix} \otimes \mathbf{q}$$

(Note that in steady state, e.g. in hover, $\vec{a} = [0 \ 0 \ -g]^\top$)

- Magnetic field quaternion representation rotated by $\mathbf{q}$:

$$\mathbf{s} = \mathbf{q}^{-1} \otimes \begin{bmatrix} 0 \\ \vec{s} \end{bmatrix} \otimes \mathbf{q}.$$

Figure 19: Quaternion formulation of the IMU models. **Top:** the kinematic equation $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}$ shows how the orientation quaternion changes over time due to angular velocity. **Middle:** the world-frame gravity reference rotated into the body frame via $\mathbf{q}^{-1}(\cdot)\mathbf{q}$ gives the predicted accelerometer reading. **Bottom:** the same rotation applies to the magnetometer reference.

The kinematic equation.

As the body rotates, the orientation quaternion $\mathbf{q}$ changes over time. The rate of change is:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}$$

where the angular velocity is embedded as a pure quaternion:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ \tilde{\omega}_x \\ \tilde{\omega}_y \\ \tilde{\omega}_z \end{bmatrix}$$

Symbol	Name	Meaning
$\dot{\mathbf{q}}$	Quaternion derivative	Rate of change of all four quaternion components simultaneously.
$\boldsymbol{\omega}$	Angular velocity (pure quaternion)	Gyroscope reading embedded with zero scalar part. In practice the bias-corrected rate: $(0, \omega_x - \omega_{x,b}, \omega_y - \omega_{y,b}, \omega_z - \omega_{z,b})$.
$\frac{1}{2}$	Half-angle factor	Comes from the $\alpha/2$ in the quaternion definition. A physical rotation rate of $\dot{\alpha}$ rad/s corresponds to a quaternion rate of $\frac{1}{2}\dot{\alpha}$.

Where the $\frac{1}{2}$ comes from

The quaternion stores $\cos(\alpha/2)$ and $\sin(\alpha/2)$, not α directly. If the body rotates at rate $\dot{\alpha}$, then $\alpha/2$ changes at rate $\dot{\alpha}/2$. Differentiating $q_0 = \cos(\alpha/2)$:

$$\dot{q}_0 = -\sin\frac{\alpha}{2} \cdot \frac{\dot{\alpha}}{2}$$

The factor $\frac{1}{2}$ emerges from the chain rule and propagates through all four components, giving $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega}$.

Discrete integration (Euler step with sample period T_s):

$$\mathbf{q}_k = \mathbf{q}_{k-1} + \frac{T_s}{2} \mathbf{q}_{k-1} \otimes \boldsymbol{\omega}_{k-1} \xrightarrow{\text{renormalise}} \mathbf{q}_k \leftarrow \frac{\mathbf{q}_k}{\|\mathbf{q}_k\|}$$

This is the **gyroscope integration step** — it propagates orientation forward using only the gyroscope. No accelerometer or magnetometer is used here. Over time it drifts due to gyroscope bias; the Madgwick/Mahoney correction steps in Sections 6 and 7 counteract that drift.

Reference vectors for correction.

The accelerometer and magnetometer read in the **body frame**, but their reference values (gravity, magnetic north) are known in the **world frame**. To compare prediction against measurement, rotate the world-frame reference into the body frame using the inverse rotation $\mathbf{q}^{-1} = \mathbf{q}^*$:

$$\mathbf{a} = \mathbf{q}^{-1} \otimes \begin{bmatrix} 0 \\ 0 \\ 0 \\ -g \end{bmatrix} \otimes \mathbf{q} \quad (\text{in steady state})$$

$$\mathbf{s} = \mathbf{q}^{-1} \otimes \begin{bmatrix} 0 \\ \bar{s}_x \\ \bar{s}_y \\ \bar{s}_z \end{bmatrix} \otimes \mathbf{q}$$

What exactly is happening — a concrete example

The filter holds a current orientation estimate $\mathbf{q}$. It uses $\mathbf{q}$ to **predict** what the accelerometer should read if that orientation is correct. It then compares that prediction to the **actual** sensor reading. Any mismatch means $\mathbf{q}$ is wrong and needs correcting.

Step 1 — the world-frame reference. Gravity in the world frame always points straight down:

$$\bar{\mathbf{a}}_{\text{world}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -g \end{bmatrix} \quad (\text{pure quaternion, NED: } z \text{ points down, specific force} = -g)$$

This is fixed and known — it never changes.

Step 2 — rotate into the body frame. The accelerometer sits on the body and reads forces in the body's own $x/y/z$ axes. To predict what it reads, take the world-frame gravity and rotate it into the body frame using $\mathbf{q}^{-1}(\cdot)\mathbf{q}$ (world $\rightarrow$ body, the reverse of Section 4.5):

$$\mathbf{a}_{\text{predicted}} = \mathbf{q}^{-1} \otimes \bar{\mathbf{a}}_{\text{world}} \otimes \mathbf{q}$$

Concrete example — drone rolled 90° to the right. The right wing is pointing straight down. In the body frame the y -axis (right) now aligns with world down. So gravity, which points down in the world, appears along the body $+y$ axis:

$$\mathbf{a}_{\text{predicted}} = \begin{bmatrix} 0 \\ 0 \\ -g \\ 0 \end{bmatrix} \rightarrow \vec{a}_{\text{predicted}} = (0, -g, 0)^\top$$

The filter therefore expects the accelerometer to read $(0, -g, 0)$ in the body frame.

Step 3 — compare to actual measurement.

$$\text{error} = \mathbf{a}_{\text{predicted}} - \mathbf{a}_{\text{measured}}$$

If the orientation estimate $\mathbf{q}$ is exactly right, the predicted and measured body-frame vectors match and the error is zero. If $\mathbf{q}$ is slightly off, the error is nonzero and tells the filter which way to rotate $\mathbf{q}$ to reduce the mismatch. The same principle applies identically to the magnetometer with magnetic north as the world-frame reference instead of gravity.

Section 5 — Key Takeaways

- All three IMU sensors follow: **measured** = **true** + **bias** + **noise**. Bias integrates into a linearly growing error ($\propto t$); noise grows as $\sqrt{t}$. Estimating and removing bias is the most critical step for long-term accuracy.
- In steady state the accelerometer reads gravity $\bar{\mathbf{a}} = (0, 0, -g)^\top$ (NED), giving a roll/pitch reference. The magnetometer gives a yaw reference. Both are unreliable

during manoeuvres or near magnetic disturbances.

- The **kinematic equation** $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega}$ propagates orientation from gyroscope data. Integrated: $\mathbf{q}_k = \mathbf{q}_{k-1} + \frac{T_s}{2}\mathbf{q}_{k-1} \otimes \boldsymbol{\omega}_{k-1}$, then renormalise. This step drifts without correction.
- World-frame references are rotated into the body frame via $\mathbf{q}^{-1} \otimes (\cdot) \otimes \mathbf{q}$ (inverse of Section 4.5's body→world rotation). The mismatch between predicted and measured body-frame vectors drives the correction in Sections 6 and 7.

Section 6 Madgwick Filter

The Madgwick filter is a computationally efficient attitude estimator specifically designed for IMU data. It is a **complementary filter operating on quaternions**: the gyroscope drives the orientation forward (high-frequency path) and the accelerometer and magnetometer correct it (low-frequency path) — exactly the complementary structure from Section 3, but applied to 3D orientation. It also estimates and removes the gyroscope bias in real time.

6.1 Algorithm Overview

- Measurements $\vec{\omega}_k$, $\vec{a}_k$ and $\vec{s}_k$ have been acquired; measurement update:

$$\begin{aligned}\hat{\mathbf{a}}_k &= \hat{\mathbf{q}}_{k-1}^{-1} \otimes \mathbf{a} \otimes \hat{\mathbf{q}}_{k-1} \\ \hat{\mathbf{s}}_k &= \hat{\mathbf{q}}_{k-1}^{-1} \otimes \mathbf{s} \otimes \hat{\mathbf{q}}_{k-1}\end{aligned}$$

where $\mathbf{a}$ and $\mathbf{s}$ are reference quaternions.

- Compute correction terms:

$$\begin{aligned}\tilde{\vec{\omega}}_k &= \vec{a}_k \times \hat{\vec{a}}_k + \vec{s}_k \times \hat{\vec{s}}_k \\ \hat{\tilde{\omega}}_{b,k} &= \hat{\tilde{\omega}}_{b,k-1} - k_i T_s \tilde{\vec{\omega}}_k\end{aligned}$$

- Estimation update:

$$\begin{aligned}\hat{\vec{\omega}}_k &= \begin{bmatrix} 0 \\ \vec{\omega}_k \end{bmatrix} - \begin{bmatrix} 0 \\ \hat{\tilde{\omega}}_{b,k} \end{bmatrix} + \begin{bmatrix} 0 \\ k_p \tilde{\vec{\omega}}_k \end{bmatrix} \\ \hat{\mathbf{q}}_k &= \hat{\mathbf{q}}_{k-1} + \frac{T_s}{2} \hat{\mathbf{q}}_{k-1} \otimes \hat{\vec{\omega}}_k.\end{aligned}$$

Figure 20: Madgwick filter update equations. **Top**: predicted body-frame readings $\hat{\mathbf{a}}_k$ and $\hat{\mathbf{s}}_k$ are computed by rotating the world-frame reference vectors into the body frame using the previous quaternion estimate $\hat{\mathbf{q}}_{k-1}$. **Middle**: cross products between measured and predicted vectors give the correction direction $\tilde{\vec{\omega}}_k$; this is integrated to estimate the gyroscope bias $\tilde{\omega}_{b,k}$. **Bottom**: the bias-corrected and proportionally adjusted angular velocity drives the quaternion forward by one step.

The filter runs three steps every time step k :

1. **Predict** what the accelerometer and magnetometer should read given the current orientation estimate.
2. **Compute the correction** from the mismatch between predicted and actual readings, and use it to update the gyroscope bias estimate.

3. **Integrate** the corrected angular velocity to advance the quaternion.

6.2 Step 1 — Predicted Body-Frame Readings

Using the previous quaternion estimate $\hat{q}_{k-1}$, rotate the world-frame reference vectors into the body frame (Section 5.2):

$$\hat{a}_k = \hat{q}_{k-1}^{-1} \otimes \mathbf{a} \otimes \hat{q}_{k-1}, \quad \hat{s}_k = \hat{q}_{k-1}^{-1} \otimes \mathbf{s} \otimes \hat{q}_{k-1}$$

where $\mathbf{a} = (0, 0, 0, -g)$ is the world-frame gravity reference and $\mathbf{s} = (0, \bar{s}_x, \bar{s}_y, \bar{s}_z)$ is the world-frame magnetic field reference (both pure quaternions).

$\hat{a}_k$ is the **predicted accelerometer reading** in the body frame — what the sensor should read if $\hat{q}_{k-1}$ is correct. $\hat{s}_k$ is the **predicted magnetometer reading**.

6.3 Step 2 — Correction Term and Bias Update

The correction direction is found from the cross products between the *measured* (bias-corrected) and *predicted* body-frame vectors:

$$\tilde{\omega}_k = \tilde{a}_k \times \hat{a}_k + \tilde{s}_k \times \hat{s}_k$$

Why the cross product gives the correction direction

The cross product $\vec{u} \times \vec{v}$ produces a vector perpendicular to both $\vec{u}$ and $\vec{v}$, with magnitude $|\vec{u}||\vec{v}|\sin\phi$ where ϕ is the angle between them.

- If the orientation estimate is **perfect**: $\tilde{a}_k$ and $\hat{a}_k$ point in the same direction, $\phi = 0$, and $\tilde{a}_k \times \hat{a}_k = \vec{0}$ — no correction needed.
- If the estimate is **slightly wrong**: the two vectors are misaligned by angle ϕ . The cross product gives a vector pointing along the rotation axis about which the body frame must be rotated to bring them back into alignment, with magnitude $\approx \phi$ for small errors.

Concrete example: Suppose the drone is actually rolled 5° right but the quaternion estimate thinks it is level. The predicted gravity $\hat{a}_k$ points straight down in the body frame (as if level), but the measured gravity $\tilde{a}_k$ is tilted 5° from down (because the body is actually tilted). The cross product $\tilde{a}_k \times \hat{a}_k$ points along the body x -axis (forward) and has magnitude $\sin 5^\circ \approx 0.087$. This tells the filter: “rotate about x to correct the roll by approximately 5° ” — which is exactly the right correction.

Adding both cross products means the accelerometer corrects roll and pitch (via gravity direction) and the magnetometer corrects yaw (via magnetic north direction). Together they constrain all three rotation axes.

The gyroscope **bias estimate** is updated by integrating the correction:

$$\tilde{\omega}_{b,k} = \tilde{\omega}_{b,k-1} - k_I T_s \tilde{\omega}_k$$

Symbol	Name	Meaning
$\tilde{\omega}_k$	Correction direction	Cross-product error signal telling the filter which way the orientation estimate needs to rotate. Zero when estimate is correct.
$\tilde{\omega}_{b,k}$	Bias estimate	Current estimate of the gyroscope drift. Updated every step by integrating the correction signal.
k_I	Integral gain	Controls how aggressively the bias estimate is updated. Large k_I : fast bias tracking but sensitive to disturbances. Small k_I : slow, smooth.
T_s	Sample period	Time between filter updates (seconds).

Why integrating the correction identifies the bias

If the gyroscope has a bias, the orientation estimate will be *systematically* wrong in the same direction at every time step. The correction term $\tilde{\omega}_k$ will therefore be persistently nonzero in the same direction. Integrating a persistent, directional signal builds up an estimate of that systematic offset — which is exactly the bias.

Once the bias estimate has converged to the true bias value, the subtraction in Step 3 cancels it and $\tilde{\omega}_k$ drops to near zero. This is the **integral** action of the filter — the same principle as the I-term in a PID controller eliminating steady-state error.

6.4 Step 3 — Corrected Angular Velocity and Quaternion Update

The corrected angular velocity combines three contributions:

$$\boldsymbol{\omega}_k = \underbrace{\begin{bmatrix} 0 \\ \tilde{\omega}_k \end{bmatrix}}_{\text{raw gyro}} - \underbrace{\begin{bmatrix} 0 \\ \tilde{\omega}_{b,k} \end{bmatrix}}_{\text{bias removal}} + \underbrace{\begin{bmatrix} 0 \\ k_P \tilde{\omega}_k \end{bmatrix}}_{\text{proportional correction}}$$

Term	Name	Meaning
$\tilde{\omega}_k$	Raw gyroscope rate	The bias-corrupted angular velocity from the gyroscope.
$-\tilde{\omega}_{b,k}$	Bias removal	Subtract the current bias estimate to get the true angular rate. This is the bias correction discussed in Section 5.
$+k_P \tilde{\omega}_k$	Proportional correction	Add a fraction of the cross-product error to the angular velocity, giving an immediate push toward the correct orientation. k_P controls how strongly the current mismatch corrects the estimate right now.

The P and I gains — PI controller on orientation error

- k_P (**proportional**): adds the current error $\tilde{\omega}_k$ directly to the angular rate — an immediate “snap back” toward the accelerometer/magnetometer reference. Large k_P = fast correction but noisy, sensitive to vibration and magnetic disturbances. Small k_P = smooth but slow to correct drift.
- k_I (**integral**): slowly accumulates the error to learn and cancel the gyroscope bias. Large k_I = learns bias fast but can become unstable if the system is frequently disturbed. Small k_I = stable, slow bias learning.

In practice $k_P \gg k_I$ — the proportional correction acts on current error while the integral must be slow enough to distinguish true gyroscope drift from temporary accelerometer/magnetometer disturbances.

The quaternion is advanced by one step using the kinematic equation:

$$\hat{q}_k = \hat{q}_{k-1} + \frac{T_s}{2} \hat{q}_{k-1} \otimes \omega_k$$

followed by renormalisation: $\hat{q}_k \leftarrow \hat{q}_k / \|\hat{q}_k\|$.

6.4b Worked Example — One Complete Filter Step

Scenario: The filter’s current estimate says the drone is perfectly level: $\hat{q}_{k-1} = (1, 0, 0, 0)$. In reality, the drone has drifted to 10° rolled to the right (about the body x -axis), but the filter does not know this yet. The gyroscope reads zero (no current rotation happening). No bias has been estimated yet: $\tilde{\omega}_{b,k-1} = (0, 0, 0)$.

Parameters: $k_P = 2.0$, $k_I = 0.005$, $T_s = 0.01$ s.

Step 1 — Predicted body-frame readings.

Since $\hat{q}_{k-1}$ is the identity, rotating any world-frame vector into the body frame leaves it unchanged:

$$\hat{a}_k = \hat{q}_{k-1}^{-1} \otimes (0, 0, 0, -1) \otimes \hat{q}_{k-1} = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}$$

(gravity normalised to 1, pointing in $-z$ body direction — the filter believes the drone is level so gravity should be straight down.)

Step 2 — Actual accelerometer reading.

The drone is actually rolled 10° right. Gravity in the body frame now has a component along body $+y$ (the right side is tilted down):

$$\tilde{a}_k = \begin{bmatrix} 0 \\ \sin 10^\circ \\ -\cos 10^\circ \end{bmatrix} = \begin{bmatrix} 0 \\ 0.174 \\ -0.985 \end{bmatrix}$$

Step 3 — Correction direction via cross product.

$$\begin{aligned}\tilde{\omega}_k &= \tilde{a}_k \times \hat{a}_k = \begin{bmatrix} 0 \\ 0.174 \\ -0.985 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 0.174 \cdot (-1) - (-0.985) \cdot 0 \\ (-0.985) \cdot 0 - 0 \cdot (-1) \\ 0 \cdot 0 - 0.174 \cdot 0 \end{bmatrix} = \begin{bmatrix} -0.174 \\ 0 \\ 0 \end{bmatrix}\end{aligned}$$

The correction points in the $-x$ direction. **Reading this:** the filter needs to rotate the estimate about the $-x$ axis (roll left) by approximately $\sin^{-1}(0.174) = 10^\circ$ — which is exactly right, since the estimate is 10° too far right.

Step 4 — Bias update.

$$\tilde{\omega}_{b,k} = (0, 0, 0) - 0.005 \times 0.01 \times (-0.174, 0, 0) = (0.0000087, 0, 0) \text{ rad/s}$$

After just one step the bias estimate is tiny — the integral builds up slowly over many steps. If the 10° roll error persists (e.g. because of a real gyroscope bias), the correction $(-0.174, 0, 0)$ will be nonzero at every step, and the integral will accumulate into a meaningful bias estimate over time.

Step 5 — Corrected angular velocity.

$$\omega_k = \underbrace{(0, 0, 0)}_{\text{gyro}} - \underbrace{(0.0000087, 0, 0)}_{\text{bias}} + \underbrace{2.0 \times (-0.174, 0, 0)}_{k_P \tilde{\omega}_k} = (-0.348, 0, 0) \text{ rad/s}$$

The dominant term is the proportional correction: -0.348 rad/s about the x -axis. This makes the filter behave as if the drone is currently rolling *left* at 0.348 rad/s — pushing the estimate back toward the true orientation.

Step 6 — Quaternion update.

Embed ω_k as a pure quaternion: $(0, -0.348, 0, 0)$.

Since $\hat{q}_{k-1} = (1, 0, 0, 0)$, the product $\hat{q}_{k-1} \otimes \omega_k = (0, -0.348, 0, 0)$.

$$\hat{q}_k = (1, 0, 0, 0) + \frac{0.01}{2}(0, -0.348, 0, 0) = (1, -0.00174, 0, 0)$$

After renormalisation: $\hat{q}_k \approx (1.000, -0.00174, 0, 0)$.

What this means: $q_x = -0.00174 = \sin(\alpha/2)$ for a small negative rotation about x . This corresponds to a $\approx 0.2^\circ$ roll to the left applied to the estimate in this one step. Starting from 0° error, after this step the estimate is 0.2° closer to the true 10° roll. After ≈ 50 steps (0.5 seconds) the estimate would catch up to the true orientation.

Summary of what changed in this one step:

Quantity	Before	After
Quaternion $\hat{q}$	(1, 0, 0, 0) — level	(1, -0.00174, 0, 0) — 0.2° roll left (toward true orientation)
Bias estimate $\tilde{\omega}_b$	(0, 0, 0)	(0.0000087, 0, 0) — tiny start of integral
Cross product error —	—	(-0.174, 0, 0) — points $-x$, says “roll left”

k_P does react to noise — and k_I is specifically why that does not ruin the bias estimate

You are right: k_P multiplies the *current* cross-product error $\tilde{\omega}_k$, which contains *both* real orientation error *and* sensor noise. A noise spike in the accelerometer produces a nonzero cross product for that one step, and k_P immediately pushes the quaternion in the wrong direction. This is an unavoidable cost of the proportional term — it cannot tell the difference between “the estimate is wrong” and “the sensor had a noise spike”. Keeping k_P small reduces this sensitivity but also slows down genuine corrections.

The integral term k_I is more robust to noise because noise is **zero-mean**: a positive spike at one step is followed by a negative spike at the next. Integrating zero-mean noise over many steps, the positive and negative contributions cancel and the integral stays near zero. But a real gyroscope bias pushes the error in the *same direction every single step* — those never cancel. Integrating a persistent, one-directional signal builds up to a significant estimate; integrating zero-mean noise stays near zero. This is exactly why k_I learns bias without being driven by noise.

Does this only work if the bias is a fixed offset?

No — it also works when the bias changes, but with an important caveat about *how fast* it changes:

- **Fixed offset, different value every startup:** works perfectly. Every time the filter starts, it begins with zero bias estimate and converges to whatever the current bias is within a few seconds. Different startup, different bias value — the integral finds it each time.
- **Slowly drifting bias** (e.g. temperature changes over minutes): still works. As the bias slowly drifts, the persistent correction signal shifts direction slowly, and the integral tracks it. As long as the bias changes *much slower* than k_I can integrate, the estimate follows it.
- **Rapidly changing bias** (changes fast compared to the integration speed k_I): the integral falls behind. The estimate always lags the true bias, causing residual drift. Increasing k_I would help track faster changes but at the cost of being more sensitive to noise — the fundamental trade-off.

In practice, gyroscope bias drifts on timescales of seconds to minutes (thermal drift, mechanical relaxation), so a slow k_I is almost always fast enough to track it while still

rejecting noise.

6.5 Flight Experiment and Performance

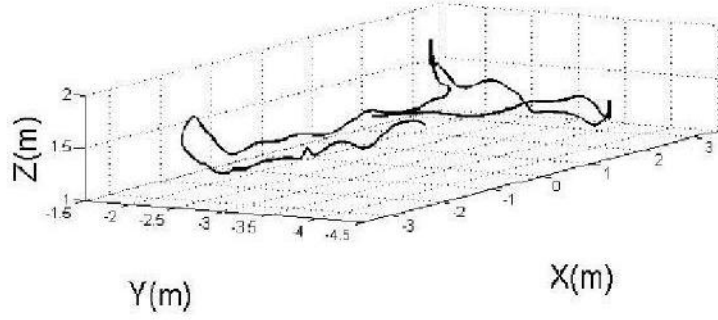


Figure 21: 3D flight trajectory during the experiment. The drone flew over a volume of roughly $5\text{ m} \times 8\text{ m} \times 1\text{ m}$. This trajectory was used to evaluate the attitude filters — the IMU data recorded here was processed by the Madgwick, Mahoney, and EKF filters and compared against a ground-truth reference.

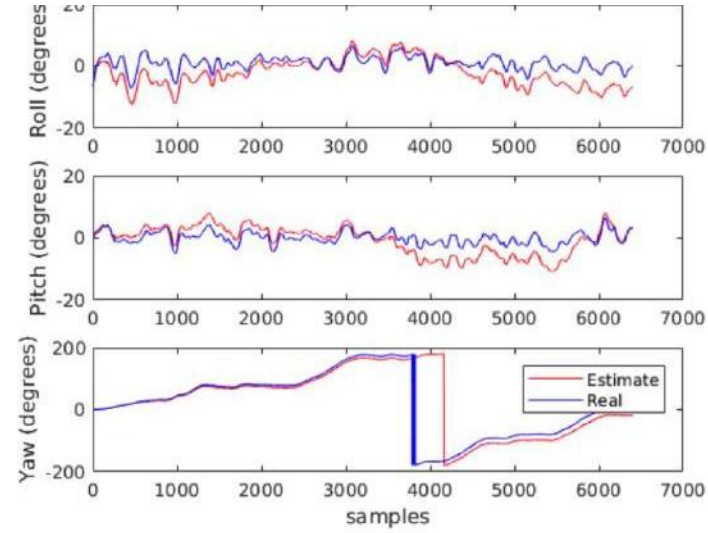


Figure 22: Madgwick filter performance. **Red:** filter estimate. **Blue:** ground truth. **Top:** roll ϕ — tracked well throughout. **Middle:** pitch θ — similarly accurate. **Bottom:** yaw ψ — larger errors, particularly around sample 3500–4000 where a rapid manoeuvre caused a temporary spike. Yaw is harder because it relies on the magnetometer, which is more susceptible to motor magnetic disturbance than the accelerometer used for roll and pitch.

Why yaw is harder than roll and pitch

Roll and pitch are corrected by the **accelerometer** via gravity. Gravity is strong ($\approx 9.81 \text{ m/s}^2$), stable, and always present.

Yaw is corrected by the **magnetometer** via magnetic north. The Earth's field is $10\text{--}60 \mu\text{T}$ — much weaker. A quadcopter's motors and speed controllers generate magnetic fields that can easily overwhelm the Earth's field near the sensor. When the motors change speed during aggressive manoeuvres, the local disturbance changes, causing spikes the filter misinterprets as yaw changes. This is visible in the bottom plot: roll and pitch track well throughout, but yaw spikes when the motors are stressed.

Section 6 — Key Takeaways

- Madgwick is a quaternion complementary filter: gyroscope integration (fast path) + accelerometer/magnetometer correction (slow path) + gyroscope bias estimation.
- **Correction:** $\tilde{\omega}_k = \tilde{a}_k \times \hat{a}_k + \tilde{s}_k \times \hat{s}_k$. Cross product = zero when estimate is correct; nonzero mismatch gives the rotation axis and approximate angle of error.
- **Bias update:** $\tilde{\omega}_{b,k} = \tilde{\omega}_{b,k-1} - k_I T_s \tilde{\omega}_k$. Persistent nonzero correction identifies and cancels gyroscope drift.
- **Quaternion update:** $\hat{q}_k = \hat{q}_{k-1} + \frac{T_s}{2} \hat{q}_{k-1} \otimes \omega_k$, with $\omega_k = \text{raw gyro} - \text{bias} + k_P$ correction. Renormalise after every step.
- $k_P \gg k_I$ in practice: proportional corrects now, integral slowly learns bias.
- Yaw harder than roll/pitch: gravity reference (accel) is strong and stable; magnetic north (mag) is weak and disturbed by motors.

Section 7 Mahoney Filter & Performance Comparison

The Mahoney filter solves exactly the same problem as Madgwick — quaternion attitude estimation from gyroscope, accelerometer and magnetometer — but uses a slightly different mathematical approach to find the correction direction. Instead of the cross product, it computes a **Jacobian-based gradient descent**: the direction in quaternion space that most steeply reduces the total sensor error. The structure is otherwise identical: gyroscope integration corrected by accelerometer and magnetometer, with bias estimation.

7.1 Magnetometer Pre-Processing

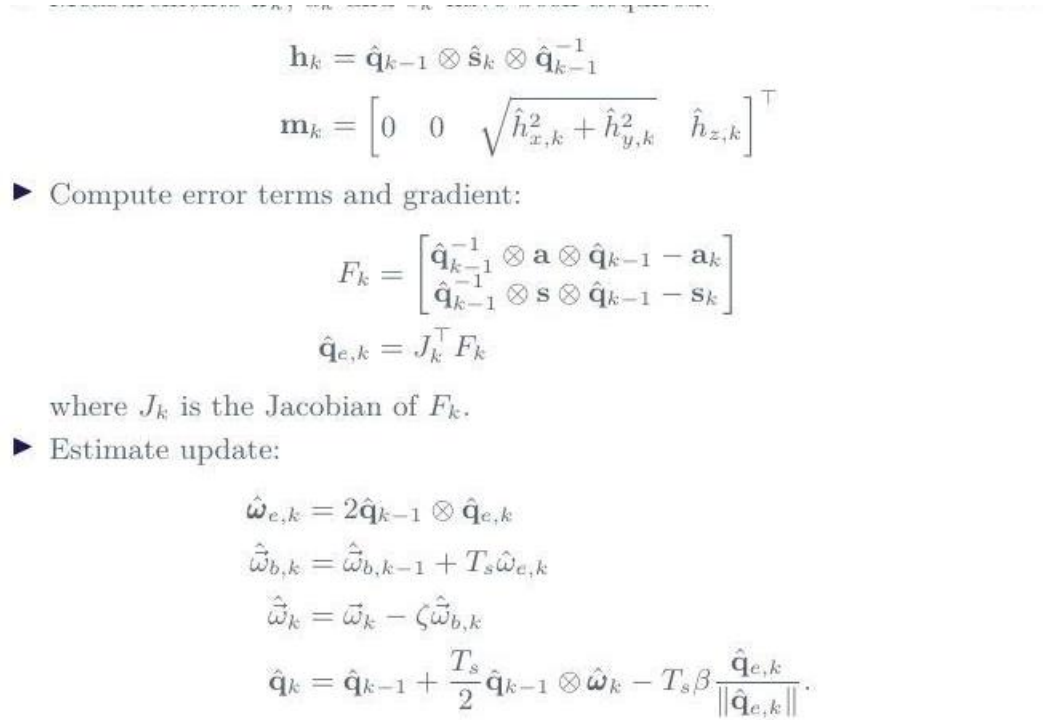


Figure 23: Mahoney filter update equations. **Top**: the magnetometer reading is rotated to the world frame ($\mathbf{h}_k$) then projected onto the horizontal plane ($\mathbf{m}_k$) to decouple yaw correction from roll/pitch. **Middle**: F_k stacks the accelerometer and magnetometer prediction errors; the Jacobian J_k maps this into quaternion space as the gradient $\hat{\mathbf{q}}_{e,k}$. **Bottom**: the gradient drives both the quaternion update and the bias estimate.

Before computing the correction, the Mahoney filter pre-processes the magnetometer reading to separate the yaw information from roll/pitch.

Rotate the measured field to the world frame:

$$\mathbf{h}_k = \hat{\mathbf{q}}_{k-1} \otimes \tilde{\mathbf{s}}_k \otimes \hat{\mathbf{q}}_{k-1}^{-1}$$

$\mathbf{h}_k = (0, h_x, h_y, h_z)$ is the estimated world-frame magnetic field.

Project onto the horizontal plane:

$$\mathbf{m}_k = \begin{bmatrix} 0 \\ \sqrt{h_{x,k}^2 + h_{y,k}^2} \\ 0 \\ h_{z,k} \end{bmatrix}$$

Why project onto the horizontal plane?

Only the **horizontal component** of the Earth's field points toward magnetic north and carries yaw information. The vertical component is the same regardless of which way you face — it tells you nothing about yaw.

The problem: when the drone is rolled or pitched, the body-frame magnetometer reading mixes horizontal and vertical components together. A 5° roll error bleeds some vertical field into the horizontal reading and vice versa — the raw body-frame reading is a corrupted yaw reference.

By first rotating the reading to the world frame ($\mathbf{h}_k$) and keeping only the horizontal magnitude $\sqrt{h_x^2 + h_y^2}$ and vertical h_z , the filter reconstructs a reference $\mathbf{m}_k$ that is **always horizontal** regardless of any roll or pitch. Yaw and roll/pitch corrections are then independent of each other.

7.2 Error, Gradient and Update Equations

Error function. Stack the prediction errors for both sensors:

$$F_k = \begin{bmatrix} \hat{q}_{k-1}^{-1} \otimes \mathbf{a} \otimes \hat{q}_{k-1} - \tilde{a}_k \\ \hat{q}_{k-1}^{-1} \otimes \mathbf{m}_k \otimes \hat{q}_{k-1} - \tilde{s}_k \end{bmatrix}$$

$F_k = \mathbf{0}$ when the orientation estimate is perfect.

Gradient in quaternion space:

$$\hat{q}_{e,k} = J_k^\top F_k$$

where J_k is the Jacobian of F_k with respect to the quaternion components. $\hat{q}_{e,k}$ is a 4-vector pointing in the direction of steepest error increase in quaternion space — the filter steps in the opposite direction.

Cross product (Madgwick) vs Jacobian gradient (Mahoney)

Both find a correction direction from the same sensor mismatch:

- **Madgwick:** $\tilde{a}_k \times \hat{a}_k$ — geometric axis perpendicular to the two vectors, magnitude $\approx \sin \phi$. Fast, no Jacobian needed, but only uses vector geometry.
- **Mahoney:** $J_k^\top F_k$ — mathematically the direction of steepest descent in the full 4D quaternion space. More principled, accounts for how each quaternion component contributes to each sensor output.

For small errors both give nearly the same correction — which is why their performance on the flight data is similar. The Mahoney gradient is theoretically more exact;

the Madgwick cross product is a fast geometric approximation.

Update equations.

Convert the quaternion gradient to an angular velocity correction:

$$\tilde{\omega}_{e,k} = 2 \hat{q}_{k-1} \otimes \hat{q}_{e,k}$$

Update bias estimate:

$$\tilde{\omega}_{b,k} = \tilde{\omega}_{b,k-1} + T_s \tilde{\omega}_{e,k}$$

Apply bias correction to gyroscope:

$$\tilde{\omega}_k \leftarrow \tilde{\omega}_k - \zeta \tilde{\omega}_{b,k}$$

Update quaternion:

$$\hat{q}_k = \hat{q}_{k-1} + \frac{T_s}{2} \hat{q}_{k-1} \otimes \tilde{\omega}_k - T_s \beta \frac{\hat{q}_{e,k}}{\|\hat{q}_{e,k}\|}$$

Symbol	Name	Meaning
$\hat{q}_{e,k}$	Quaternion gradient	Steepest ascent direction in quaternion space. Filter steps against it.
$\tilde{\omega}_{e,k}$	Angular correction	Gradient converted to equivalent angular velocity for bias update.
β	Gradient gain	Controls how strongly the normalised gradient corrects the quaternion. Equivalent to k_P in Madgwick.
ζ	Bias gain	Controls how strongly the bias estimate is applied. Equivalent to k_I in Madgwick.
$\hat{q}_{e,k}/\ \hat{q}_{e,k}\ $	Normalised gradient	The gradient is normalised before applying: step size is fixed at βT_s regardless of error magnitude. Madgwick's cross product is <i>not</i> normalised — larger errors give larger automatic corrections.

7.2b Worked Example — One Complete Mahoney Step

Same scenario as the Madgwick example: $\hat{q}_{k-1} = (1, 0, 0, 0)$, drone drifted 10° rolled right, gyroscope reads zero, zero initial bias. Parameters: $\beta = 2.0$, $\zeta = 0.005$, $T_s = 0.01$ s.

Step 1 — Magnetometer pre-processing.

Suppose the magnetometer reads $\tilde{s}_k = (0.7, 0.5, 0.3)$ (body frame, after bias removal, normalised for simplicity) and the reference world-frame magnetic field is known as $\bar{s} = (1, 0, 0.3)$ (pointing mostly North with a vertical dip component).

Rotate measured field to world frame:

$$\mathbf{h}_k = \hat{q}_{k-1} \otimes \tilde{s}_k \otimes \hat{q}_{k-1}^{-1}$$

At identity, this leaves $\tilde{s}_k$ unchanged: $\mathbf{h}_k = (0, 0.7, 0.5, 0.3)$.

Project onto horizontal plane (keep horizontal magnitude and vertical component, zero out the other horizontal component):

$$\mathbf{m}_k = \begin{bmatrix} 0 \\ \sqrt{0.7^2 + 0.5^2} \\ 0 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.860 \\ 0 \\ 0.3 \end{bmatrix}$$

This is now the reference used for yaw correction — it is always horizontal regardless of any roll or pitch in the estimate.

Step 2 — Predicted body-frame readings (same as Madgwick).

At identity: $\hat{a}_k = (0, 0, -1)$ (gravity straight down in body).

Step 3 — Error function F_k .

Using the same accelerometer reading as the Madgwick example: $\tilde{a}_k = (0, 0.174, -0.985)$.

$$F_{acc} = \hat{a}_k - \tilde{a}_k = (0, 0, -1) - (0, 0.174, -0.985) = (0, -0.174, -0.015)$$

(For the magnetometer part we skip the computation for brevity; the accelerometer already captures the main roll correction.)

Step 4 — Jacobian J_k at the identity quaternion.

For the accelerometer function $f_{acc}(\mathbf{q}) = \hat{q}^{-1} \otimes (0, 0, 0, -1) \otimes \hat{q}$, the Jacobian evaluated at $\hat{q} = (1, 0, 0, 0)$ is:

$$J_{acc} = \left. \frac{\partial f_{acc}}{\partial (q_0, q_x, q_y, q_z)} \right|_{(1,0,0,0)} = \begin{bmatrix} 0 & 0 & 2 & 0 \\ 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \end{bmatrix}$$

Each column is how a small nudge in that quaternion component changes the predicted gravity vector. Column q_x ($\partial f / \partial q_x = (0, -2, 0)$) says: a small positive roll pushes the predicted y -component of gravity downward (more negative).

Step 5 — Gradient $\hat{q}_{e,k} = J_k^\top F_k$.

$$J_{acc}^\top = \begin{bmatrix} 0 & 0 & -2 \\ 0 & -2 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad F_{acc} = \begin{bmatrix} 0 \\ -0.174 \\ -0.015 \end{bmatrix}$$

$$\hat{q}_e = J_{acc}^\top F_{acc} = \begin{bmatrix} 0(0) + 0(-0.174) + (-2)(-0.015) \\ 0(0) + (-2)(-0.174) + 0(-0.015) \\ 2(0) + 0(-0.174) + 0(-0.015) \\ 0 \end{bmatrix} = \begin{bmatrix} 0.03 \\ 0.348 \\ 0 \\ 0 \end{bmatrix} \approx \begin{bmatrix} 0 \\ 0.348 \\ 0 \\ 0 \end{bmatrix}$$

The gradient points in the $+q_x$ direction — the same correction axis as the Madgwick cross product.

Step 6 — Normalise and apply the gradient.

$$\|\hat{q}_e\| \approx 0.348, \quad \frac{\hat{q}_e}{\|\hat{q}_e\|} \approx (0, 1, 0, 0)$$

$$\text{Gradient correction step} = -T_s \beta \frac{\hat{q}_e}{\|\hat{q}_e\|} = -0.01 \times 2.0 \times (0, 1, 0, 0) = (0, -0.02, 0, 0)$$

Step 7 — Bias update.

$$\tilde{\omega}_e = 2\hat{q}_{k-1} \otimes \hat{q}_e = 2(1, 0, 0, 0) \otimes (0, 0.348, 0, 0) = (0, 0.696, 0, 0)$$

$$\tilde{\omega}_b = (0, 0, 0) + T_s \tilde{\omega}_e = 0.01 \times (0, 0.696, 0, 0) = (0, 0.00696, 0, 0)$$

Apply bias: $\tilde{\omega}_k \leftarrow (0, 0, 0) - \zeta \tilde{\omega}_b = -0.005 \times (0, 0.00696, 0, 0) \approx (0, 0, 0)$ (negligible this step).

Step 8 — Quaternion update.

$$\hat{q}_k = \underbrace{(1, 0, 0, 0)}_{\hat{q}_{k-1}} + \underbrace{0}_{\text{gyro term (gyro}=0)} + \underbrace{(0, -0.02, 0, 0)}_{\text{gradient step}} = (1, -0.02, 0, 0)$$

After renormalisation: $\hat{q}_k \approx (0.9998, -0.02, 0, 0)$.

Comparing Madgwick and Mahoney on the same step:

	Madgwick	Mahoney	Same direction?
Correction method	Cross product $\tilde{a} \times \hat{a}$	Jacobian $J^T F$	Yes
Correction direction	$(-0.174, 0, 0)$ angular rate	$(0, -0.02, 0, 0)$ quaternion	Yes (both $-q_x$)
q_x after update	$+0.00174$	-0.02	Same sign
Correction size/step	$\propto \sin \phi$ (scales with error)	$= \beta T_s = 0.02$ (fixed)	Different

Both methods move the quaternion in the same direction. The key difference is the **normalisation**: Mahoney always takes a step of size $\beta T_s = 0.02$ regardless of how large the error is; Madgwick takes a step proportional to $\sin \phi \approx 0.174$, so small errors produce small corrections automatically.

7.3 Performance and Comparison with EKF

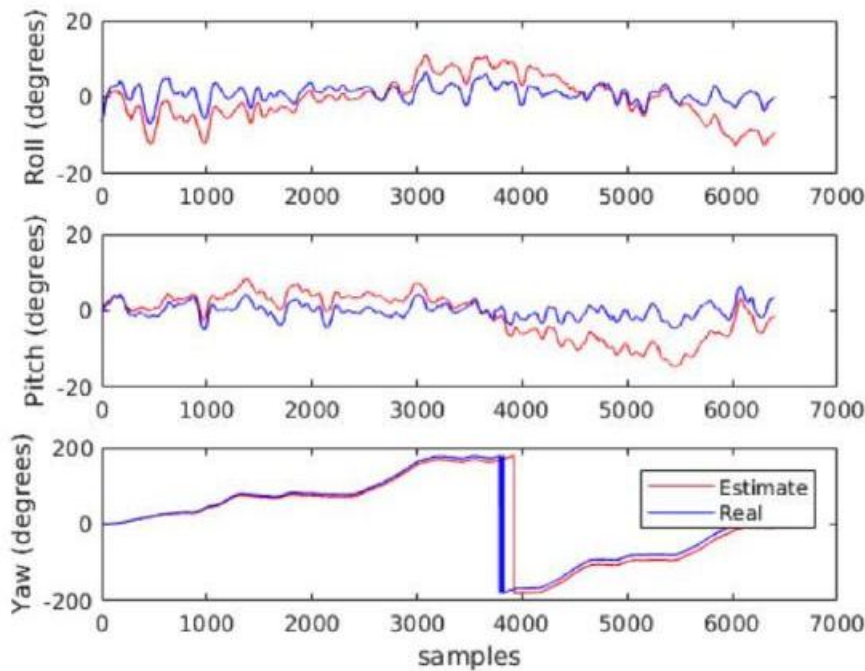


Figure 24: Mahoney filter performance. **Red:** filter estimate. **Blue:** ground truth. Roll and pitch are well tracked throughout. Yaw shows the same spike at sample 3500–4000 as Madgwick (same magnetometer disturbance, same cause). Outside the spike, Mahoney’s general yaw tracking is slightly cleaner than Madgwick’s — a direct benefit of the horizontal plane projection in Section 7.1, which prevents roll/pitch estimation errors from bleeding into the yaw correction. Madgwick uses the raw body-frame magnetometer reading, so any small tilt error in the estimate contaminates the yaw correction; Mahoney’s projection decouples them.

The yaw problem is a sensor issue, not a filter issue

Madgwick, Mahoney, and the EKF below all show the same yaw spike at the same moment. This is because all three rely on the **magnetometer** for yaw correction, and the magnetometer is disturbed at that moment by the drone’s own motors.

Roll and pitch use **gravity** as a reference — strong (9.81 m/s^2), constant, unaffected by electronics or manoeuvres. Yaw uses the **Earth’s magnetic field** — weak ($\sim 50 \mu\text{T}$), easily overwhelmed by motor and ESC currents which change every time the drone manoeuvres.

Changing the filter algorithm does not help because the information was never in the sensor reading to begin with. The fix is hardware: mount the magnetometer far from the motors, or use a different yaw reference entirely (GPS heading, optical flow, external tracking system).

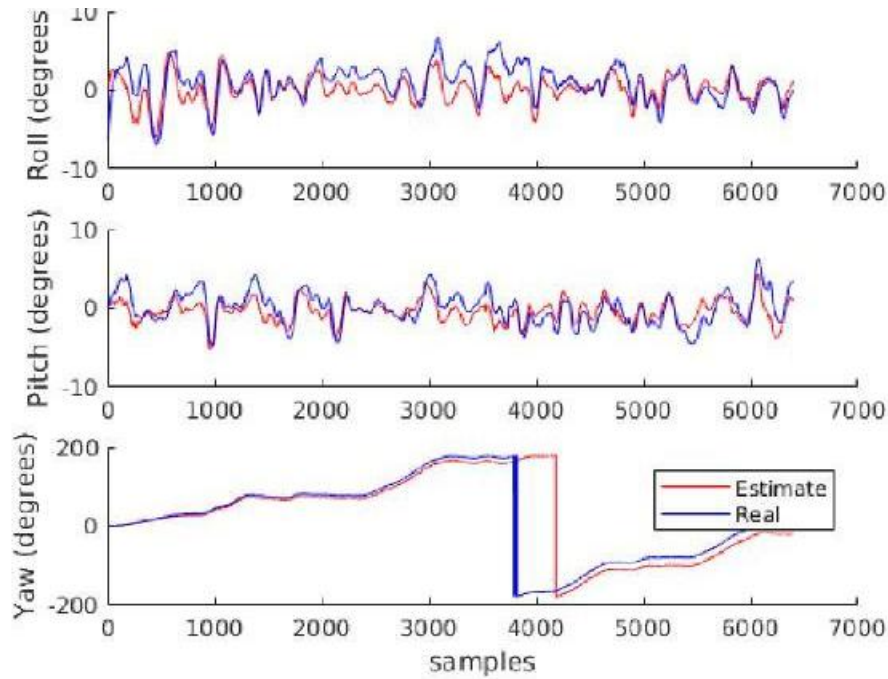


Figure 25: Extended Kalman Filter performance on the same flight data. Roll and pitch are tracked more accurately than both Madgwick and Mahoney. Yaw still spikes at the same location — confirming this is a magnetometer disturbance that no filter algorithm can overcome. The EKF achieves better accuracy at significantly higher computational cost.

Madgwick vs Mahoney vs EKF — when to use which

	Madgwick	Mahoney	EKF
Correction method	Cross product	Jacobian gradient	Kalman gain (optimal)
Noise model	None — hand-tuned k_P, k_I	None — hand-tuned β, ζ	Explicit Q, R covariances
CPU cost	Very low	Low	High
Roll/pitch	Good	Good	Best
Yaw	Limited by magnetometer — same for all three		
Best for	Microcontrollers, drones	Same	High-accuracy systems

All three share the same fundamental structure. The EKF is statistically optimal given a correct noise model; Madgwick and Mahoney replace the noise model with tunable gains, which is simpler and cheaper but less principled. For most embedded drone applications Madgwick or Mahoney is the practical choice.

Section 7 — Key Takeaways

- Mahoney uses **Jacobian gradient descent** $\hat{q}_{e,k} = J_k^\top F_k$ rather than cross product. Similar performance in practice; more principled mathematically.
- The magnetometer is **projected onto the horizontal plane** ($\mathbf{m}_k$) to decouple yaw correction from roll/pitch. Prevents tilt errors from contaminating the yaw estimate.
- Gains: β = gradient correction strength (like k_P); ζ = bias learning rate (like k_I). Gradient is **normalised** before applying — fixed step size unlike Madgwick's unnormalised cross product.
- **The yaw spike is identical in all three filters.** It is a magnetometer hardware problem, not a filter design problem. No algorithm can recover information the sensor never contained.
- Choose Madgwick/Mahoney for embedded low-CPU use with hand-tuned gains; choose EKF when accuracy is critical and a noise model is available.

Lecture Summary — Sensor Fusion from Kalman to Attitude Filters

This lecture developed sensor fusion from its most general form down to three specific algorithms, each making progressively more specialised assumptions. Here is the connecting thread.

1 — Kalman Filter for Multi-Sensor Fusion (Section 2)

The most general approach. The challenge is that multiple sensors can have **correlated noise** between them — if two sensors share a power supply, a common clock, or the same environmental disturbance, their errors are not independent. Directly applying the standard KF ignores this and produces an overconfident, suboptimal estimate.

The fix is the **Cholesky decorrelation**: factorise $R_k = L_k^\top L_k$, then transform all measurements by $T_k = (L_k^{-1})^\top$. This produces transformed channels with identity covariance $\bar{R}_k = I$ — fully independent, unit variance. The standard KF update then applies unchanged.

Importantly, the same R_k matrix that performs the decorrelation also serves a second role: it **encodes how much to trust each sensor**. The diagonal entries R_{ii} are the noise variances of individual sensors. Through the transformation $T_k = (L_k^{-1})^\top$, these variances are directly applied to the observation matrix $\bar{H}_k = T_k H_k$ and the transformed measurements $\bar{y}_k = T_k y_k$. A sensor with large variance gets divided by a large factor — its contribution to the update is scaled down. A sensor with small variance gets less scaling — its contribution dominates. This means R_k simultaneously **removes correlations** and **determines which sensors the filter relies on and by how much** at each time step. Both effects flow directly into the Kalman gain K_k through $\bar{H}_k$ and $\bar{R}_k$, making the weighting automatic and statistically optimal.

This is what none of the complementary filters below can match: they have no equivalent of R_k and cannot adapt their sensor weighting based on measured noise levels.

2 — Complementary Filter (Section 3)

A simpler alternative when two sensors split naturally by frequency: one is reliable at low frequencies (e.g. GPS position, steady-state gravity), the other at high frequencies (e.g. velocity, gyroscope rate). The complementary filter assigns the low-frequency band to one sensor and the high-frequency band to the other via $F_1(s) + F_2(s) = 1$, so the full spectrum is covered exactly once.

The critical limitation compared to the Kalman filter: the split is controlled by a **single fixed time constant** τ . τ sets the crossover frequency but cannot adapt to changing noise levels. The KF automatically adjusts its sensor weighting as noise conditions change (e.g. GPS becomes less reliable near buildings); the complementary filter simply applies the same fixed τ regardless. It is therefore optimal only when sensor noise levels are constant — which in practice they often are not.

3 — Madgwick Filter (Section 6)

A complementary filter applied to 3D orientation using quaternions. The gyroscope is the high-frequency path (integrates fast rotations); the accelerometer and magnetometer are the low-frequency correction path (provide stable gravity and magnetic north references).

The correction direction is found via the **cross product** $\tilde{a}_k \times \hat{a}_k$, which is *not normalised*: a larger angular error between predicted and measured gravity produces a proportionally larger correction step. This is an advantage for **fast or large changes**: when the estimate suddenly drifts far from reality, the cross product is large and Madgwick snaps back quickly. The downside is that a large noise spike also produces a large (wrong) correction, since the filter cannot tell drift from noise.

Gyroscope bias is estimated by integrating the correction signal (the k_I integral term).

4 — Mahoney Filter (Section 7)

Structurally identical to Madgwick but uses a **Jacobian-based normalised gradient** instead of the raw cross product. Crucially, the gradient is normalised before applying: the correction step is always exactly βT_s regardless of how large the error is.

This has two consequences:

- **Slower recovery from large sudden errors**: if the estimate jumps far from reality in one step, Mahoney applies the same fixed-size correction step as for a small error. It cannot snap back as fast as Madgwick, so it may temporarily fall behind during rapid manoeuvres.
- **Better steady-state yaw**: Mahoney pre-processes the magnetometer by rotating its reading to the world frame and projecting onto the horizontal plane before using it as a reference. This decouples yaw correction from any residual roll/pitch estimation errors — a roll error in the estimate does not contaminate the yaw correction. Madgwick uses the raw body-frame magnetometer reading directly, so small tilt errors bleed into yaw. The result: Mahoney's steady-state yaw tracking is slightly cleaner than Madgwick's.

The magnetometer is inherently the hardest reference to use for yaw because the Earth's field ($\sim 50 \mu\text{T}$) is weak and easily overwhelmed by the drone's own motor currents, which change constantly during flight. This is why all three filters show the same yaw spike at the same moment — it is a **sensor hardware problem**, not an algorithm problem. No filter design can recover information the magnetometer never contained.

5 — Overall Comparison: Why the EKF Tracks Roll and Pitch Better

Comparing the three filters side by side on the same flight data, the **EKF achieves the best roll and pitch accuracy**. The reason goes back to the fundamental

advantage identified in point 1: the EKF has an **explicit noise model**.

The covariances Q and R describe exactly how much to trust the gyroscope integration versus the accelerometer correction at every step. The Kalman gain is computed to minimise the mean-squared estimation error given those models. When the accelerometer is reliable (low acceleration), it gets more weight; when it is disturbed (high acceleration), it gets less. This continuous, quantitative adaptation is what the complementary filters cannot do.

Madgwick and Mahoney replace the noise model with hand-tuned gains k_P/k_I or β/ζ . Once set, these gains are fixed: they cannot distinguish between “the gyroscope is drifting slowly today because it is warm” and “the accelerometer is unreliable right now because the drone is thrusting hard.” The EKF adapts; the complementary filters do not.

The trade-off is computational cost: the EKF requires matrix inversions and covariance propagation at every step, which is expensive on embedded hardware. For most drone applications where CPU is limited and accuracy requirements are moderate, Madgwick or Mahoney is the practical choice. When accuracy is critical and a full noise model is available, the EKF is superior.

Why EKF and not the standard KF? The standard KF only works for *linear* systems. The attitude problem is nonlinear in two places:

- **State dynamics:** the kinematic equation $\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega}$ involves a *product* of the state $\mathbf{q}$ with the gyroscope input $\boldsymbol{\omega}$. Multiplying the state by an input is nonlinear — the standard KF assumes the state update is $x_{k+1} = \Phi x_k + \Gamma u_k$ (state and input enter separately, never multiplied).
- **Measurement model:** the predicted sensor readings $\hat{q}^{-1} \otimes \bar{a} \otimes \hat{q}$ involve quaternion products — the state $\mathbf{q}$ appears multiplied by itself, which is quadratic (nonlinear) in the state components.

The EKF handles this by **linearising** both equations around the current estimate at each step using Jacobians — exactly the same idea as the EKF seen in earlier lectures. A standard KF applied directly would give wrong results because it assumes these relationships are linear when they are not.